



TABLE OF CONTENTS / INDEX

S.No.	Topic / Section Title	Handwritten Pages
1	History of Java & Core Features	Pages 1 – 5
2	JVM, JRE, JDK & Program Generations	Pages 6 – 9
3	Input Handling: Scanner, BigInteger & BigDecimal	Pages 10 – 14
4	Java Expressions, Data Types & Flow Control	Pages 15 – 18
5	Control Flow Statements (If-Else, Switch, Loops)	Pages 19 – 23
6	Classes, Objects & Constructors	Pages 24 – 32
7	Object-Oriented Programming (OOPs) Concepts	Pages 33 – 42
8	Inheritance, Super Keyword & Method Overriding	Pages 43 – 52
9	Polymorphism, Abstraction & Abstract Classes	Pages 53 – 62
10	Interfaces, Default/Static Methods & Packages	Pages 63 – 72
11	Exception Handling (Try-Catch, Custom Exceptions)	Pages 73 – 85
12	String Handling, StringBuilder & StringBuffer	Pages 86 – 100
13	Enums, Static Keyword & JVM Memory Model	Pages 101 – 117

[Handwritten Notes Page 1]

History of Java

- 1. **James Gosling, Mike Sheridan, and Patrick Naughton** initiated the Java language project in **June 1991**.
- 2. The small team of Sun engineers was called the **Green Team**.
- 3. First, it was called "**Greentalk**" by James Gosling, and the file extension was **.gt**.
- 4. Java was originally designed for small, embedded systems in electronic appliances like set-top boxes, but it was too advanced technology for the digital cable television industry at the time.
- 5. After that, it was called **Oak** and was developed as a part of the Green Project.
- 6. Later, Java technology was incorporated by **Netscape** and suited for networking.
- 7. Java was called **Oak** as it is a symbol of strength and chosen as the national tree of many countries like U.S.A., France, Germany, Romania, etc.
- 8. In **1995**, Oak was renamed as **Java**.
- - Java is an island of Indonesia where the first coffee was produced (called Java coffee).
- 9. **1995** → *Time* magazine called Java one of the **Ten Best Products of 1995**.
- 10. **JDK 1.0** (Java Development Kit) was released on **January 23, 1996**.
- 11. Java programming is characterized as:
 - - Simple
 - - Portable
 - - Secure
 - - High-Performance
 - - Multithreaded
 - - Interpreted
 - - Platform Independent
 - - Dynamic
 - - Architecture-Neutral
 - - Object-Oriented
 - - Robust
- 12. Java provides platform independence, offers backward compatibility, serves as the base for the Android mobile platform, and supports parallel and distributed development.

[Handwritten Notes Page 2]

JAVA (3rd Generation Programming Language)

- - To save the file, the extension is **.java**.
- 1. Java can be considered as both a **compiled** and an **interpreted** language because its source code is first compiled into binary code (bytecode).
- 2. This bytecode runs on the **Java Virtual Machine (JVM)**, which is usually a software-based interpreter.
- 3. To run any program in Java, we need the **JRE (Java Runtime Environment)**.

- - **JDK (Java Development Kit)** → Contains both **JVM & JRE**.

NOTE:

When we compile code having the extension **.java**:

- - After compiling, it converts into bytecode having the extension **.class**. Each individual class is put in its own output **.class** file.

Basic Structure of a Java Program:

```
class Main {
    public static void main(String[] args) {
        // Code goes here
    }
}
```

Programming Language Generations & Execution Flow:

- - **1st Generation**: Machine level (**100100**)
- - **2nd Generation**: Assembly level (**ADD XY, MOV ZA**) → Translated via **Assembler**
- - **3rd Generation (3GL)**: High level (**main() { ... }**)
- - **Interpreter**: Runs code line-by-line / one-by-one.
- - **Compiler**: Converts entire code into machine level at once.
- - **4th Generation (4GL)**: SQL

Commonly known Third Generation Programming Languages (3GL):

Fortran, BASIC, Pascal, Java, C, C++, C#.

[\[Handwritten Notes Page 3 \]](#)

Writing Output in Java (print vs println)

Syntax for Print Method:

```
System.out.print("Hello, World!");
```

Example 1:

```
public class Main {
    public static void main(String[] args) {
        System.out.print("Hello, World!");
    }
}
```

NOTE:

In the above program, ensure that:

- 1. In **System.out.print()**:
 - - **System** is a built-in class (**S** should be capitalized).
 - - **out** is the standard output stream object.
- 2. In **String**: **S** is capitalized.
- 3. **class <Name>**: Must match the **.java** file name if the class is public.

Example 2:

```
class Main {
    public static void main(String[] args) {
        System.out.println("Hello, World!"); // Line 1
        System.out.print("Hello, World!");   // Line 2
    }
}
```

Output:

```
Hello, World!
Hello, World!
```

- - In **Line 1**, **println** prints the text and automatically moves the cursor to the next line.
- - In **Line 2**, **print** prints the text and keeps the cursor on the same line.

Java Comments:

- - **//** → Single-line comment
- - **/* ... */** → Multi-line comment (like C language)

[Handwritten Notes Page 4]

Variables, Identifiers, Literals & Data Types

Variable:

A variable is a location in memory used to hold data.

Identifiers:

Identifiers are names used to identify programming elements (variables, methods, classes, etc.).

Literals:

Literals are data values used for representing fixed constants directly in the code.

- - Examples: `int a = 1;`, `float b = 2.5f;`, `char c = 'F';`

Number System Literals:

- - Binary: `int binaryNumber = 0b10010;`
- - Octal: `int octalNumber = 027;`
- - Decimal: `int decNumber = 34;`
- - Hexadecimal: `int hexNumber = 0x2F;`

Java Data Types (8 Primitive Types)

1. `boolean` Type

- 1. The `boolean` data type has only two possible values: `true` or `false`.
- 2. The default value is `false`.
- 3. They are usually used for conditional logic and flag checks.

Example:

```
class Main {
    public static void main(String[] args) {
        boolean flag = true;
        System.out.print(flag);
    }
}
```

Output:

```
true
```

[[Handwritten Notes Page 5](#)]

Primitive Data Types (Integer Types)

2. `byte` Type (8-bit signed 2's complement integer)

- 1. The `byte` data type can have values from `-128` to `127`.
- 2. If it is certain that the value of a variable will be within `-128` to `127`, it is used instead of `int` to save memory.
- 3. **Default value:** `0`

Example:

```
class Main {
    public static void main(String[] args) {
        byte range;
        range = 124;
        System.out.print(range);
    }
}
```

Output: **124**

3. `short` Type (16-bit signed 2's complement integer)

- 1. The **short** data type in Java can have values from **-32,768 to 32,767**.
- 2. If it is certain that the value will be within -32768 to 32767, it is used instead of **int** or **long**.
- 3. **Default value: 0**

4. `int` Type (32-bit signed 2's complement integer)

- 1. Can have values from **-2^{31} to $2^{31} - 1$** (-2,147,483,648 to 2,147,483,647).
- 2. **Default value: 0**

5. `long` Type (64-bit signed 2's complement integer)

- 1. Can have values from **-2^{63} to $2^{63} - 1$** .
- 2. **Default value: 0L**
- - **Example:**

```
long range = -4233322000L; // 'L' suffix denotes long literal
```

[[Handwritten Notes Page 6](#)]

Primitive Data Types (Floating-Point & Char) & Strings

6. `double` Type (64-bit IEEE 754 Floating Point)

- 1. The **double** data type stores decimal values with double precision.
- 2. It should never be used for precise values such as currency.
- - **Range:** $\$4.9 \times 10^{-324}$ to $\$1.8 \times 10^{+308}$
- - **Default value: 0.0d** (or **0.0**)

7. `float` Type (32-bit IEEE 754 Floating Point)

- 1. Stores single-precision decimal values.
- 2. It should never be used for precise values such as currency.
- - **Range:** $\$1.4 \times 10^{-45}$ to $\$3.4 \times 10^{+38}$
- - **Default value:** `0.0f`

Example:

```
class Main {
    public static void main(String[] args) {
        float number = -42.3f;
        System.out.print(number);
    }
}
```

8. `char` Type (16-bit Unicode Character)

- - **Range:** `0` to `65535`
- 1. Stores characters enclosed in single quotation marks (' ').
- 2. The minimum value is `'\u0000'` and the maximum value is `'\uffff'`. These are Unicode representations.

Example:

```
class Main {
    public static void main(String[] args) {
        char letter = '\u0051';
        System.out.print(letter);
    }
}
```

Output: `Q` *(Unicode code point \u0051 is the letter 'Q')*

String (Reference Type):

```
String str = "Hello";
```

- - A sequence of characters enclosed in double quotation marks (" ") is called a **String**.

[\[Handwritten Notes Page 7 \]](#)

Java Operators & Console Input

1. Arithmetic Operators:

`+`, `-`, `*`, `/`, `%`

2. Assignment Operators:

`=, +=, -=, *=, /=, %=`

- - `a += b` (equivalent to `a = a + b`)
- - `a -= b` (equivalent to `a = a - b`)
- - `a *= b` (equivalent to `a = a * b`)

3. Relational Operators:

`==, !=, >, <, >=, <=`

- - Used to check relationships between two operands.

4. Logical Operators:

- - `&&` (Logical AND): Evaluates to `true` if both conditions are true.
- - `||` (Logical OR): Evaluates to `true` if any one condition is true.
- - `!` (Logical NOT): Evaluates to `true` if expression is false.

5. Java Unary Operators:

Unary operators are used with only one operand:

- - `++` (Increment)
- - `--` (Decrement)
- - `!` (Logical Complement)
- - `-` (Unary Minus: inverts sign of expression)

6. Bitwise Operators:

- - `~` → Bitwise Complement
- - `&` → Bitwise AND
- - `^` → Bitwise Exclusive OR (XOR)
- - `|` → Bitwise OR
- - `<<` → Left Shift
- - `>>` → Right Shift
- - `>>>` → Unsigned Right Shift

Ternary Operator:

```
variable = Expression ? Expression1 : Expression2;
```

- - If `Expression` is `true`, `Expression1` is evaluated and assigned.
- - If `Expression` is `false`, `Expression2` is evaluated and assigned.

Java Input (Scanner):

To take user input:

- 1. Import the **Scanner** class:

```
import java.util.Scanner;
```

- 2. Create an object of **Scanner**:

```
Scanner input = new Scanner(System.in);
```

- 3. Take input from the user:

```
int number = input.nextInt();
```

[[Handwritten Notes Page 8](#)]

Scanner Class & Command Line Arguments

Scanner is a class in the **java.util** package used to read input data from different sources like input streams (**System.in**), files, etc.

Example:

```
import java.util.Scanner;

class Main {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        System.out.print("Enter your Name: ");
        String name = input.nextLine();

        System.out.println("My name is " + name);

        input.close();
    }
}
```

Output:

```
Enter your Name: Kelvin
My name is Kelvin
```

Command Line Arguments:

- - Compile: **javac FileName.java**
- - Run: **java FileName arg1 arg2**
- - *Example*: **java AllDatatypes 2 Virat**

Explanation:

In `Scanner input = new Scanner(System.in);`:

- 1. We created an object of `Scanner` named `input`.
- 2. The `System.in` parameter is used to read from the standard input stream (keyboard).
- 3. We use the `nextLine()` method of the `Scanner` class to read a complete line of text from the user.

[Handwritten Notes Page 9]

Java Scanner Methods for User Input

Method	Description
<code>nextInt()</code>	Reads an <code>int</code> value from the user
<code>nextFloat()</code>	Reads a <code>float</code> value from the user
<code>nextBoolean()</code>	Reads a <code>boolean</code> value from the user
<code>nextLine()</code>	Reads a full line of text from the user
<code>next()</code>	Reads a single word/token from the user
<code>nextByte()</code>	Reads a <code>byte</code> value from the user
<code>nextDouble()</code>	Reads a <code>double</code> value from the user
<code>nextShort()</code>	Reads a <code>short</code> value from the user
<code>nextLong()</code>	Reads a <code>long</code> value from the user

| :--- | :--- |

| `nextInt()` | Reads an `int` value from the user |

| `nextFloat()` | Reads a `float` value from the user |

| `nextBoolean()` | Reads a `boolean` value from the user |

| `nextLine()` | Reads a full line of text from the user |

| `next()` | Reads a single word/token from the user |

| `nextByte()` | Reads a `byte` value from the user |

| `nextDouble()` | Reads a `double` value from the user |

| `nextShort()` | Reads a `short` value from the user |

| `nextLong()` | Reads a `long` value from the user |

NOTE:

In the above Scanner methods:

- 1. **No space** between `next` and the type keyword:
 - - `next Line` ■
 - - `nextLine` ✓
- 2. **CamelCase formatting**: After `next`, the data type must start with an uppercase letter:
 - - `nextDouble()` ✓ | `nextdouble()` ■
 - - `nextBoolean()` ✓ | `nextboolean()` ■
- 3. **`next()` behavior**:
 - - Reads characters up to whitespace.
 - - When whitespace is encountered, it returns the single token/word.

[Handwritten Notes Page 10]

Java Scanner with BigInteger and BigDecimal

Reading Large Numbers:

- - `nextBigInteger()` → Reads a **BigInteger** value from the user.
- - `nextBigDecimal()` → Reads a **BigDecimal** value from the user. *(No space in method name)*

To use them, first import:

```
import java.math.BigInteger;
import java.math.BigDecimal;
```

BigInteger Arithmetic Methods:

- - `num1.add(num2)`
- - `num1.subtract(num2)`
- - `num1.divide(num2)`
- - `num1.multiply(num2)`
- - `num1.remainder(num2)`
- - `b1.pow(exponent)`
- - `boolean response = b1.equals(b2);`

> [!NOTE]

> We cannot add **BigInteger** objects using the `+` operator. They must be operated on using methods: `.add()`, `.subtract()`, `.multiply()`, `.divide()`.

`compareTo()` Method:

```
b1.compareTo(b2);
```

- - Returns **0** if `b_1 == b_2`
- - Returns **1** if `b_1 > b_2`
- - Returns **-1** if `b_1 < b_2`

Example:

```
import java.math.BigInteger;
import java.util.Scanner;

class Main {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        System.out.print("Enter a big integer: ");
        BigInteger value1 = input.nextBigInteger();

        System.out.println("Using nextBigInteger(): " + value1);

        input.close();
    }
}
```

*(Similarly for **BigDecimal** using **input.nextBigDecimal();**)*

Java Expression:

A Java expression consists of variables, operators, literals, and method calls that evaluate to a single value.

- - **score = 90** is an expression.
- - In **if (numb1 == numb2)**, **numb1 == numb2** is an expression.

[[Handwritten Notes Page 11](#)]

Java Statements and Flow Controls

Java Statements:

In Java, each statement is a complete unit of execution.

```
int score = 9 * 5;
```

Here, we have a statement. The complete execution involves multiplying integers 9 and 5, and then assigning the result to the variable **score**. In the above statement, **9 * 5** is an expression.

NOTE:

In Java, expressions are part of statements. We can convert an expression into a statement by terminating it with a semicolon (;). These are known as **expression statements**.

- - Expression: **number = 10**
- - Statement: **number = 10;**

Control Statements:

- - Looping Statements
- - Branching / Selection / Conditional Statements

JAVA FLOW CONTROLS:

1. **if** (if-then) statement:

```
if (condition) {
    // statements
}
```

2. **if...else** (if-then-else) statement:

```
if (condition) {
    // codes in if block
} else {
    // codes in else block
}
```

3. **if...else if** (else-if ladder):

```
if (condition1) {
    // code
} else if (condition2) {
    // code
} else {
    // code
}
```

[[Handwritten Notes Page 12](#)]

Nested Conditions & Java Loops

4. Nested **if...else**:

```
if (condition1) {
    if (condition2) {
        // code
    } else {
        // code
    }
} else {
    // code
}
```

Nested Ternary Operator:

```
int max = (n1 >= n2) ? ((n1 >= n3) ? n1 : n3) : ((n2 >= n3) ? n2 : n3);
```

Loops in Java:

1. **for** Loop:

```
for (initialExpression; testExpression; updateExpression) {
    // body of the loop
}
```

Multiple Variables in Loop Header:

```
for (int a = 1, b = 4; a < b; a++, b--) {
    System.out.println("a = " + a);
    System.out.println("b = " + b);
}
```

2. **while** Loop:

```
// initialization
while (testExpression) {
    // body of loop
    // increment / decrement
}
```

3. **do...while** Loop:

```
do {
    // body of loop
} while (condition);
```

4. For-Each Loop (Enhanced **for** loop):

Syntax:

```
for (dataType item : array) {
    // body of loop
}
```

Example:

```
class Main {
    public static void main(String[] args) {
        int[] numbers = {3, 9, 5, -5};
        for (int number : numbers) {
            System.out.println(number);
        }
    }
}
```

Java `break` and Labeled `break` Statement

The **break** statement is used to terminate loop execution prematurely.

- - Syntax: **break;**

Java Labeled `break` Statement:

Used to terminate an outer named loop from within an inner loop.

Syntax:

```
while (testExpression) {
    second:
    while (testExpression) {
        // codes
        while (testExpression) {
            // codes
            break second;
        }
    }
    // control jumps here
}
```

Example Code:

```
class LabeledBreak {
    public static void main(String[] args) {
        first:
        for (int i = 1; i < 5; i++) {
            second:
            for (int j = 1; j < 3; j++) {
                System.out.println("i = " + i + "; j = " + j);
                if (i == 2) {
                    break first;
                }
            }
        }
    }
}
```

Output:

```
i = 1; j = 1
i = 1; j = 2
i = 2; j = 1
```

*(When $i = 2$ and $j = 1$, **break first** executes, terminating the outer loop immediately).*

Java `continue` and `switch` Statements

Java `continue` Statement:

The `continue` statement skips the current iteration of a loop.

After the `continue` statement, control moves to the loop's update step or condition.

- - Syntax: `continue;`

Labeled `continue` Statement:

- - Syntax: `continue labelName;`

Example:

```
class Main {
    public static void main(String[] args) {
        first:
        for (int i = 1; i < 6; ++i) {
            for (int j = 1; j < 5; ++j) {
                if (i == 3 || j == 2) {
                    continue first;
                }
                System.out.println("i = " + i + "; j = " + j);
            }
        }
    }
}
```

Output:

```
i = 1; j = 1
i = 2; j = 1
i = 4; j = 1
i = 5; j = 1
```

Java `switch` Statement:

```
switch (expression) {
    case value1:
        // code
        break;
    case value2:
        // code
        break;
    default:
        // code
}
```

[\[Handwritten Notes Page 15 \]](#)

Java Classes

A **class** in Java is a set of objects that share common characteristics, behaviors, properties, and attributes.

- - **Attributes** means fields / member variables: They are variables declared within a class and represent the properties of an object created from the class.
- - A class is a user-defined blueprint or prototype from which objects are created.
- - *Example*: **Student** is a class, while a particular student named "Ravi" is an object.

Creating a Class in Java:

We can create a class in Java using the **class** keyword:

```
class ClassName {  
    // fields (variables to store data)  
    // methods  
}
```

Example:

```
class Bicycle {  
    private int gear = 5;  
  
    public void braking() {  
        System.out.println("Working of Braking");  
    }  
}
```

In the above example, we created a class named **Bicycle**. It contains a field named **gear** and a method named **braking()**.

[\[Handwritten Notes Page 16 \]](#)

Properties & Components of Java Classes

In the bicycle example, **Bicycle** serves as a prototype (an original model/template).

Now, we can create any number of bicycle objects using this prototype, and all bicycles will share its fields and methods.

Properties of Java Classes:

- 1. A class is not a real-world physical entity; it is just a template, blueprint, or prototype from which objects are created.
- 2. A class declaration does not occupy memory (memory is allocated when objects are created).
- 3. A class is a collection of variables of different data types and a group of methods.
- 4. A class in Java can contain:
 - 1. Data members (fields)
 - 2. Methods
 - 3. Constructors
 - 4. Nested classes
 - 5. Interfaces

Components of a Java Class:

In general, class declarations include these components in order:

- 1. **Modifiers:** A class can be **public** or have package-private (default) access.
- 2. **class keyword:** Used to declare a class.
- 3. **Class Name:** Starts with a capital letter by naming conventions (PascalCase).
- 4. **Body:** Surrounded by curly braces { }.

[[Handwritten Notes Page 17](#)]

Java Objects

An **object** is an instance of a class (also known as a class object or class instance).

Accessing Members of a Class:

```
class Bicycle {
    int gear = 5;

    void braking() {
        // ...
    }
}
```

Instantiating and accessing:

```
Bicycle sportsBicycle = new Bicycle(); // Object creation
sportsBicycle.gear;           // Accessing field
sportsBicycle.braking();      // Accessing method
```

Example: `Box` Class

```
class Box {
    double width;
    double height;
    double depth;
}

class BoxDemo {
    public static void main(String[] args) {
        Box mybox = new Box();
        double vol;
        mybox.width = 10;

        Box mybox1 = new Box(10, 20, 15);
        // 10, 20, 15 assigned to width, height, depth respectively
    }
}
```

Summary:

- - In the example above, we created a class named **Bicycle** containing a field **gear** and a method **braking()**.
- - We created an object of **Bicycle** named **sportsBicycle**.
- - We used the dot (.) operator with the object to access the fields and methods of the class.
- - By creating a single object, we can access any number of fields and methods. There is no need to create a separate object for each field/member.

[[Handwritten Notes Page 18](#)]

Java Methods

A **method** is a block of code that performs a specific task when called.

Types of Methods in Java:

- 1. **User-defined Methods**: Created by the programmer to meet specific application requirements.
- 2. **Standard Library Methods**: Built-in methods provided by Java (e.g., **Math.sqrt()**, **System.out.println()**).

Declaring a Java Method (Basic Syntax):

```
returnType methodName() {
    // method body
}
```

Components:

- - **returnType**: Specifies what type of value the method returns. For example, if a method has an **int** return type, it returns an integer value. If the method does not return a value, its return type is **void**.
- - **methodName**: An identifier used to refer to and call the method in a program.
- - **method body**: Includes the statements executed to perform the task.

Example:

```
int addNumbers() {
    // code
}
```

[[Handwritten Notes Page 19](#)]

Method Declaration & Invocation

Complete Syntax of Declaring a Method:

```
modifier static returnType nameOfMethod(param1, param2) {
    // method body
}
```

- - **Modifier**: Defines access level (e.g., **public**, **private**, **protected**, or default).
- - **static**: When present, allows the method to be accessed without creating an instance/object of the class.
- - **Parameters**: Values passed into the method.

Calling a Method in Java:

```
addNumbers();
```

Example:

```
class Main {
    public int addNumbers(int a, int b) {
        int sum = a + b;
        return sum;
    }

    public static void main(String[] args) {
        int num1 = 25;
        int num2 = 15;

        Main obj = new Main();
        int result = obj.addNumbers(num1, num2);
        System.out.println("Sum is: " + result);
    }
}
```

Output:

```
Sum is: 40
```

[Handwritten Notes Page 20]

Static vs Instance Methods & Java Method Overloading

- - In the previous example, `addNumbers()` is **not static**, so an object was created (`new Main()`) to call it.
- - If a method is declared **static** and called within the same class, it can be invoked directly without creating an object or referencing the class name.

JAVA METHOD OVERLOADING

In Java, two or more methods can share the same name if they differ in parameter lists (different number of parameters, different types of parameters, or both).

- - These methods are called **overloaded methods**, and this feature is called **Method Overloading**.

Example Signatures:

```
void func() { ... }
void func(int a) { ... }
float func(double a) { ... }
float func(int a, float b) { ... }
```

Here, `func()` is overloaded because each version accepts different arguments.

Example 1: Overloading by Changing the Number of Parameters:

```
class MethodOverloading {
    private static void display(int a) {
        System.out.println("Arguments: " + a);
    }

    private static void display(int a, int b) {
        System.out.println("Arguments: " + a + " and " + b);
    }

    public static void main(String[] args) {
        display(1);
        display(1, 4);
    }
}
```

Output:

```
Arguments: 1
Arguments: 1 and 4
```

[\[Handwritten Notes Page 21 \]](#)

2. Method Overloading by Changing the Data Type of Parameters

```
class MethodOverloading {  
    private static void display(int a) {  
        System.out.println("Got Integer data.");  
    }  
  
    private static void display(String a) {  
        System.out.println("Got String object.");  
    }  
  
    public static void main(String[] args) {  
        display(1);  
        display("Hello");  
    }  
}
```

Output:

```
Got Integer data.  
Got String object.
```

Explanation:

Here both overloaded methods accept one argument. However, one accepts an argument of type **int**, whereas the other accepts a **String** object.

> [!NOTE]

> ★ It is **not** method overloading if we only change the return type of methods. There must be a difference in the number of parameters or the data types of parameters.

[\[Handwritten Notes Page 22 \]](#)

Java Constructors

- 1. A constructor in Java is similar to a method that is **invoked** when an object of a class is created.
- - *Invoke* means to execute / call a method.
- 2. Unlike Java methods, a constructor has the **same name as that of the class** and **does not have any return type**.

Example:

```
class Test {  
    Test() {  
        // constructor body  
    }  
}
```

Here `Test()` is a constructor. It has the same name as that of the class, and doesn't have a return type.

Types of Constructors

In Java, constructors can be divided into three types:

- 1. **No-Arg Constructor**
- 2. **Parameterized Constructor**
- 3. **Default Constructor**

1. Java No-Arg Constructors

Similar to methods, a Java constructor may or may not have any parameters.

If a constructor does not accept any parameters, it is known as a **no-argument constructor** (no-arg constructor).

Example Syntax:

```
private Constructor() {  
    // body of the constructor  
}
```

[[Handwritten Notes Page 23](#)]

Example: Java Private No-Arg Constructor

```
class Main {  
    int i;  
  
    // Constructor with no parameters  
    private Main() {  
        i = 5;  
        System.out.println("Constructor is called");  
    }  
  
    public static void main(String[] args) {  
        Main obj = new Main();  
        System.out.println("Value of i: " + obj.i);  
    }  
}
```

Output:

```
Constructor is called  
Value of i: 5
```

Key Notes:

- - Here the constructor does not accept any parameters. Hence, it is known as a **no-arg constructor**.
- - Once a constructor is declared **private**, it cannot be accessed from outside of the class.

- - So, we are creating the object inside the same class.

Example: Java Public No-Arg Constructor

```
class Company {
    String name;

    public Company() {
        name = "Programiz";
    }
}

class Main {
    public static void main(String[] args) {
        Company obj = new Company();
        System.out.println("Company name = " + obj.name);
    }
}
```

Output:

```
Company name = Programiz
```

Note:

Since the constructor is **public**, we can access it from another class.

[[Handwritten Notes Page 24](#)]

2. Java Parameterized Constructor

A Java constructor can also accept one or more parameters. Such constructors are known as **parameterized constructors**.

Example:

```
class Main {
    String languages;

    // Constructor accepting a single value
    Main(String lang) {
        languages = lang;
        System.out.println(languages + " Programming Language");
    }

    public static void main(String[] args) {
        Main obj1 = new Main("Java");
        Main obj2 = new Main("Python");
        Main obj3 = new Main("C");
    }
}
```

Output:

```
Java Programming Language
Python Programming Language
C Programming Language
```

Key Notes:

- - In the above example, we have created a constructor named `Main(String lang)`.
- - Here, the constructor takes a single parameter.
- - We are passing a single value argument to the constructor during object instantiation.
- - Based on the argument passed, the `languages` instance variable is initialized inside the constructor.

[[Handwritten Notes Page 25](#)]

3. Java Default Constructor

If we do not create any constructor, the Java compiler automatically creates a no-arg constructor during program compilation.

This constructor is called the **default constructor**.

Example: Default Constructor

```
class Main {
    int a;
    boolean b;

    public static void main(String[] args) {
        Main obj = new Main();
        System.out.println("Default value:");
        System.out.println("a = " + obj.a);
        System.out.println("b = " + obj.b);
    }
}
```

Output:

```
Default value:
a = 0
b = false
```

Key Notes:

- - Here, we haven't created any constructor in class `Main`.
- - Hence, the Java compiler automatically creates the default constructor.
- - The default constructor initializes uninitialized instance variables with their default values (e.g., `0` for integer types, `false` for boolean, `null` for objects).

> [!NOTE]

> **Instance Variable:** A variable that is defined in a class in Object-Oriented Programming, declared inside the class but outside of any constructors, methods, or blocks.

Important Notes on Java Constructors

- - Constructors are invoked implicitly when you instantiate objects... *(continued on next page)*

[Handwritten Notes Page 26]

Important Notes on Java Constructors (Continued)

- 1. Constructors are invoked (i.e. called) implicitly (automatically) when you instantiate (i.e. create) objects.
- 2. The two primary rules for creating a constructor are:
 - 1. The name of the constructor must be the same as the class name.
 - 2. A Java constructor must not have any return type (not even **void**).
- 3. If a class does not define a constructor, the Java compiler automatically generates a default no-argument constructor.
- 4. The default constructor initializes instance variables with their default values.
- 5. A constructor cannot be **abstract**, **static**, or **final**.

Java `static` Keyword

- 1. In Java, if we want to access class members (variables and methods), we normally must first create an instance (object) of the class.
- 2. But there will be situations where we want to access class members without creating an object.
- 3. In those situations, we can use the **static** keyword in Java.
- 4. If we want to access class members without creating an instance of the class, we declare those class members as **static**.

[Handwritten Notes Page 27]

Case 1: Accessing Static Members in the Same Class

If a static variable or method is accessed/called within the same class, there is no need to call it using the class name and no need to create any object.

Example:

```
class Practice {
    static int a = 1;

    public static void main(String[] args) {
        System.out.println(a);
    }
}
```

Output:

1

[Handwritten Notes Page 28]

Case 2: Non-Static Members Cannot Be Accessed Directly in Static Methods

Non-static methods and instance variables cannot be referenced directly inside static methods without creating an object.

Example (Causes Compilation Error):

```
class Practice {
    int a = 1;

    public static void main(String[] args) {
        System.out.println(a); // Compilation Error
    }
}
```

Output / Error:

Error: non-static variable a cannot be referenced from a static context

Case 3: Accessing Non-Static Members via Object Creation

If **static** is not used for a variable declared in a class (making it an instance variable), an object must be instantiated to access that variable.

Example:

```
class Practice {
    int a = 1;

    public static void main(String[] args) {
        Practice obj = new Practice();
        System.out.println(obj.a);
    }
}
```

Output:

1

[Handwritten Notes Page 29]

Case 4: Calling a Static Method in the Same Class

If we create a static method, we can call it directly without using the class name or creating an object, provided the call is made within the same class.

Example:

```
class Practice {
    static void vini() {
        int a = 1;
        System.out.println(a);
    }

    public static void main(String[] args) {
        vini();
    }
}
```

Output:

```
1
```

Case 5: Calling a Static Method from Another Class

If we create a static method in another class and want to call it, we must call it using the class name:

ClassName.methodName().

- - If the method is not static, you must create an object of that class to access it.

Example:

```
class Gnana {
    static int vini() {
        int a = 1;
        return a;
    }
}

class Practice {
    public static void main(String[] args) {
        System.out.println(Gnana.vini());
    }
}
```

Output:

```
1
```

[\[Handwritten Notes Page 30 \]](#)

Static Blocks

In Java, static blocks (static initialization blocks) are used to initialize static variables.

Syntax:

```
static {  
    // variable initialization  
}
```

Example:

```
class Test {  
    static int age;  
  
    static {  
        age = 23;  
    }  
}
```

> [!NOTE]

> **Note 1:** Static variables should be declared outside the static block and can be initialized either outside or inside the static block.

Key Characteristics:

- - A static block is executed **only once** when the class is loaded into memory.
- - A class is loaded when an object of the class is instantiated or any static member of the class is accessed.
- - A class can have multiple static blocks. Each static block is executed in the order (sequence) it appears in the program.

> [!NOTE]

> **Note 2:** If a variable is declared inside a static block, its scope is local to that block and it cannot be accessed outside the static block.

Example with Multiple Static Blocks:

```
class Gnana {
    static int a = 23;
    static int b;
    static int max;

    static {
        System.out.println("First Static block");
        b = a * 4;
    }

    static {
        System.out.println("Second Static block");
        max = 30;
    }

    static void display() {
        System.out.println("a = " + a);
        System.out.println("b = " + b);
        System.out.println("max = " + max);
    }
}
```

(Execution continued on next page...)

[\[Handwritten Notes Page 31 \]](#)

Static Blocks (Continued)

Main Class Execution:

```
class Practice {
    public static void main(String[] args) {
        System.out.println("Hello");
        System.out.println(Gnana.b);
    }
}
```

Output:

```
Hello
First Static block
Second Static block
92
```

Detailed Execution Walkthrough:

- 1. When **Practice.main()** begins, **System.out.println("Hello");** executes first, printing **Hello**.
- 2. Next, **Gnana.b** is referenced, which triggers the JVM to load the **Gnana** class into memory.
- 3. Upon class loading:
 - - Static variable **a** is initialized to **23**.
 - - The first static block executes: prints **"First Static block"** and calculates **b = 23 * 4 = 92**.
 - - The second static block executes: prints **"Second Static block"** and sets **max = 30**.

- 4. Finally, the value of `Gnana.b (92)` is printed.

Important Note Points

- - In Java, a source file is officially called a **compilation unit**. It is a text file that contains one or more class definitions.
- - If a class is declared **public**, the name of that class must match the name of the file holding the program (`ClassName.java`).
- - The Java compiler will successfully compile classes even if they do not contain a `main()` method.
- - If you accidentally typed `Main` instead of `main`, the compiler would still compile your program without errors.

[Handwritten Notes Page 32]

Note on `main()` Method (Continued)

- - While the compiler compiles code without a valid `main()`, the **Java Interpreter (JVM)** will report a runtime error at execution because it will be unable to find the standard entry point `public static void main(String[] args)`.

Separators in Java

Java uses specific punctuation characters known as separators (punctuators):

Separator	Name	Description
<code>{</code> , <code>}</code> , <code>(</code> , <code>)</code> , <code>[</code> , <code>]</code> , <code>:</code> , <code>;</code> , <code>,</code> , <code>.</code>	Punctuators	Used to separate code elements
<code>(</code> , <code>)</code>	Parentheses	Used to contain lists of parameters in method definitions and invocations. Also used in control statements (e.g., <code>if</code> , <code>for</code> , <code>while</code>) and for expression precedence.
<code>{</code> , <code>}</code>	Braces	Used to contain the values of automatically initialized arrays. Also used to define blocks of code for classes, methods, and local scopes.
<code>[</code> , <code>]</code>	Brackets	Used to declare array types. Also used when dereferencing array elements (indexing).
<code>;</code>	Semicolon	Statement terminator; ends a statement.
<code>,</code>	Comma	Separates consecutive identifiers in variable declarations. Also used to chain statements together inside a <code>for</code> loop update expression.
<code>.</code>	Period	Used to separate package names from subpackages and classes, and to access members/methods of an object or class.

| `{` | `}` | `(` | `)` | `[` | `]` | `:` | `;` | `,` | `.` |

| `(` | `)` | **Parentheses** | Used to contain lists of parameters in method definitions and invocations. Also used in control statements (e.g., `if`, `for`, `while`) and for expression precedence. |

| `{` | `}` | **Braces** | Used to contain the values of automatically initialized arrays. Also used to define blocks of code for classes, methods, and local scopes. |

| `[` | `]` | **Brackets** | Used to declare array types. Also used when dereferencing array elements (indexing). |

| `;` | **Semicolon** | Statement terminator; ends a statement. |

| `,` | **Comma** | Separates consecutive identifiers in variable declarations. Also used to chain statements together inside a `for` loop update expression. |

| `.` | **Period** | Used to separate package names from subpackages and classes, and to access members/methods of an object or class. |

Dynamic Initialization

In Java, variables can be dynamically initialized at runtime using any valid expression.

Example:

```
class DynInit {  
    public static void main(String[] args) {  
        double a = 3.0, b = 4.0;  
        // c is dynamically initialized using Math.sqrt()  
        double c = Math.sqrt(a * a + b * b);  
        System.out.println("Hypotenuse is " + c);  
    }  
}
```

Output:

```
Hypotenuse is 5.0
```

Notes:

- - **a** and **b** are initialized using literals/constants.
- - **c** is **dynamically initialized** at runtime based on the outcome of the mathematical expression.

[[Handwritten Notes Page 33](#)]

Type Conversion and Casting

Type conversion is the process of converting data of one data type into another.

There are two primary types of conversion in Java:

- 1. **Implicit Conversion (Automatic Conversion / Widening)**
- 2. **Explicit Conversion (Narrowing / Casting)**

1. Automatic Type Conversion (Widening Conversion)

When one type of data is assigned to another type of variable, automatic type conversion takes place if the following two conditions are met:

- 1. **The two types are compatible.**

(Compatible: Able to exist or convert smoothly without data format conflicts.)

- 2. **The destination type is larger than the source type.**

When both conditions are met, a **Widening Conversion** takes place.

What is Widening Conversion?

In Java, widening conversion refers to the automatic conversion of a smaller data type to a larger data type without any loss of precision or information. It is also known as **Implicit Conversion**.

Examples:

- - The **int** type is always large enough to hold all valid **byte** values (**int** is 32-bit, **byte** is 8-bit).
- - Numeric types (**byte** $\rightarrow$ **short** $\rightarrow$ **int** $\rightarrow$ **long** $\rightarrow$ **float** $\rightarrow$ **double**) are mutually compatible for widening.
- - **char** and **boolean** are **not compatible** with each other. **boolean** cannot be converted to or from any other primitive type.

```
int numb = 124; // Valid assignment
```

[Handwritten Notes Page 34]

Casting Incompatible Types (Narrowing Conversion)

- - It is possible to assign a **byte** value to an **int** automatically (widening).
- - But it is **not possible** to assign an **int** value to a **byte** automatically because **byte** has smaller storage capacity (8 bits) than **int** (32 bits).
- - This kind of conversion is called **Narrowing Conversion**.
- - **Narrowing Conversion**: The process of converting a larger data type into a smaller data type. Because this can result in data loss or precision loss, it must be performed explicitly by the programmer using a cast operator (**target-type** **value**).
- - Since the programmer applies the cast explicitly, any data loss is acknowledged.

Examples:

```
int a = 100;
byte b;
b = (byte) a; // Explicit cast
```

```
byte b;
int i = 257;
double d = 323.142;

b = (byte) i;           // i % 256 = 1 -> b = 1
b = (byte) d;           // Truncated to 323, 323 % 256 = 67 -> b = 67
```

Automatic Type Promotion in Expressions

Consider the following example:

```
byte a = 40;
byte b = 50;
byte c = 100;
int d = a * b / c;
```

- - The sub-expression **a * b** evaluates to **2000**, which easily exceeds the maximum range of a **byte** (-128 to 127).

- - To handle this intermediate overflow problem, Java automatically promotes each **byte**, **short**, or **char** operand to **int** when evaluating an expression.

[Handwritten Notes Page 35]

Automatic Type Promotion (Continued)

Even though operands are **byte** values, Java automatically promotes them to **int** during arithmetic evaluation.

Example:

```
byte b = 50;  
b = b * 2; // Error! Cannot assign an int to a byte!
```

Explanation:

- - The code attempts to store **50 * 2 = 100** (which fits within the byte range -128 to 127) back into **b**.
- - However, because the operands were promoted to **int** during multiplication, the resulting value is of type **int**.
- - An **int** cannot be assigned to a **byte** without an explicit cast:

```
b = (byte)(b * 2); // Correct
```

The Type Promotion Rules

When evaluating expressions, Java applies the following type promotion rules:

1. All **byte**, **short**, and **char** values are promoted to **int**.
2. If one operand is a **long**, the whole expression is promoted to **long**.
3. If one operand is a **float**, the entire expression is promoted to **float**.
4. If any of the operands is a **double**, the result of the expression is **double**.

Examples:

```
byte b = 42;  
short s = 1024;  
float f = 5.67f;  
long n = 980;  
  
float result1 = n / f; // n is promoted to float, result is float  
long result2 = n / s; // s is promoted to long, result is long
```

[Handwritten Notes Page 36]

Arrays

- - An array is a collection/group of **homogeneous elements** (elements of the same data type) stored under a single variable name.

- - Arrays of any type can be created and may have one or more dimensions.

Array Declaration:

```
int month_days[];
```

Alternative Declaration Syntaxes:

```
int a1[] = new int[3];
int[] a2 = new int[3]; // Recommended standard syntax
```

Allocating Memory for an Array:

```
month_days = new int[12];
```

Multidimensional Arrays (2-Dimensional Arrays)

```
int twoD[][] = new int[4][5]; // 4 rows, 5 columns
```

> [!NOTE]

> When you allocate memory for a multidimensional array, you only need to specify the memory for the first (leftmost) dimension. You can allocate the remaining dimensions separately, allowing for non-rectangular (ragged/jagged) arrays.

Example (Non-Uniform / Ragged Array Allocation):

```
int twoD[][] = new int[4][];
twoD[0] = new int[1];
twoD[1] = new int[2];
twoD[2] = new int[3];
twoD[3] = new int[4];
```

Pictorial Representation:

```
Row 0: [0][0]
Row 1: [1][0] [1][1]
Row 2: [2][0] [2][1] [2][2]
Row 3: [3][0] [3][1] [3][2] [3][3]
```

[[Handwritten Notes Page 37](#)]

Java Operator Precedence

Operators are listed in order from highest precedence to lowest:

Level	Operators	Description
-------	-----------	-------------

| :---: | :--- | :--- |

| 1 | **() [] .** | Parentheses, Array subscript, Member access |

| 2 | **++ -- ~ !** | Post/Pre-increment, Post/Pre-decrement, Bitwise NOT, Logical NOT |

| 3 | *** / %** | Multiplicative (Multiplication, Division, Modulo) |

| 4 | **+ -** | Additive (Addition, Subtraction) |

| 5 | **>> >>> <<** | Shift operators (Signed right, Unsigned right, Left shift) |

| 6 | **> >= < <=** | Relational operators |

| 7 | **= = ! =** | Equality operators |

| 8 | **&** | Bitwise AND / Boolean logical AND |

| 9 | **^** | Bitwise XOR / Boolean logical XOR |

| 10 | **|** | Bitwise OR / Boolean logical OR |

| 11 | **&&** | Logical AND (Short-circuit) |

| 12 | **||** | Logical OR (Short-circuit) |

| 13 | **?:** | Ternary conditional |

| 14 | **= op=** | Assignment and shorthand compound assignments (**+=**, **-=**, etc.) |

Overloading Constructors

Similar to method overloading, we can define two or more constructors within the same class that share the same name but have different parameter lists. This is known as **Constructor Overloading**.

Example:

```
class Box {
    double width;
    double height;
    double depth;

    // Constructor used when all dimensions are specified
    Box(double w, double h, double d) {
        width = w;
        height = h;
        depth = d;
    }
}
```

(Continued on next page...)

[\[Handwritten Notes Page 38 \]](#)

Constructor Overloading (Continued)

```
// Constructor used when no dimensions are specified
Box() {
    width = -1;
    height = -1;
    depth = -1;
}

// Constructor used when a cube is created (all sides equal)
Box(double len) {
    width = height = depth = len;
}

// Method to compute and return volume
double volume() {
    return width * height * depth;
}

}

class OverloadCons {
    public static void main(String[] args) {
        Box mybox1 = new Box(10, 20, 15);
        Box mybox2 = new Box();
        Box mycube = new Box(7);

        double vol;

        vol = mybox1.volume();
        System.out.println("Volume of mybox1 is " + vol);

        vol = mybox2.volume();
        System.out.println("Volume of mybox2 is " + vol);

        vol = mycube.volume();
        System.out.println("Volume of mycube is " + vol);
    }
}
```

Output:

```
Volume of mybox1 is 3000.0
Volume of mybox2 is -1.0
Volume of mycube is 343.0
```

Explanation:

Here, the **Box** class provides multiple overloaded constructor variations. The appropriate constructor is chosen based on the number and types of arguments supplied at object creation.

[\[Handwritten Notes Page 39 \]](#)

Using Objects as Parameters

In Java, it is common and practical to pass objects as parameters to methods and constructors.

Example:

```
class Test {
    int a, b;

    Test(int i, int j) {
        a = i;
        b = j;
    }

    // Return true if specified Test object has identical dimensions
    boolean equals(Test o) {
        if (o.a == a && o.b == b)
            return true;
        else
            return false;
    }
}

class PassOb {
    public static void main(String[] args) {
        Test ob1 = new Test(100, 22);
        Test ob2 = new Test(100, 22);
        Test ob3 = new Test(-1, -1);

        System.out.println("ob1 == ob2: " + ob1.equals(ob2)); // Line 1
        System.out.println("ob1 == ob3: " + ob1.equals(ob3)); // Line 2
    }
}
```

Output:

```
ob1 == ob2: true
ob1 == ob3: false
```

Concept Breakdown:

- - In Line 1:

The `equals` method is invoked on `ob1` with `ob2` passed as an argument. The values of `ob1` are accessible via `a` and `b`, while `ob2`'s reference is passed as `o`. Since `o.a == a` (`100 == 100`) and `o.b == b` (`22 == 22`), the method returns `true`.

- - In Line 2:

`ob3` is passed to `ob1.equals(ob3)`. Comparing values reveals they differ, so the method returns `false`.

[[Handwritten Notes Page 40](#)]

Example 2: Passing an Object to a Constructor (Copy Constructor Pattern)

One practical use of object parameters is creating a constructor that clones/duplicates another object of the same class.

```

class Box {
    double width;
    double height;
    double depth;

    // Constructor that clones another Box object
    Box(Box ob) { // Pass object reference to constructor
        width = ob.width;
        height = ob.height;
        depth = ob.depth;
    }

    // Constructor used when all dimensions specified
    Box(double w, double h, double d) {
        width = w;
        height = h;
        depth = d;
    }

    // Constructor used when no dimensions specified
    Box() {
        width = -1;
        height = -1;
        depth = -1;
    }

    // Constructor used when a cube is created
    Box(double len) {
        width = height = depth = len;
    }

    double volume() {
        return width * height * depth;
    }
}

class OverloadCons2 {
    public static void main(String[] args) {
        Box mybox1 = new Box(10, 20, 15);
        Box mybox2 = new Box();
        Box mycube = new Box(7);

        // Passing reference of mybox1 object to clone it
        Box myclone = new Box(mybox1);

        double vol;
        vol = mybox1.volume();
        System.out.println("Volume of mybox1 is " + vol);

        vol = myclone.volume();
        System.out.println("Volume of myclone is " + vol);
    }
}

```

Output:

```

Volume of mybox1 is 3000.0
Volume of myclone is 3000.0

```

Note:

myclone is initialized with the exact dimensions of **mybox1** by passing the reference of **mybox1** into **Box(Box ob)**.

[Handwritten Notes Page 41]

```
vol = mybox1.volume();
System.out.println("Volume of mybox1 is " + vol);

vol = mybox2.volume();
System.out.println("Volume of mybox2 is " + vol);

vol = mycube.volume();
System.out.println("Volume of cube is " + vol);

vol = myclone.volume();
System.out.println("Volume of clone is " + vol);
}
```

Output:

```
Volume of mybox1 is 3000.0 
Volume of mybox2 is -1.0 
Volume of clone is 3000.0 
```

Same (mybox1 and myclone have same volume)

Returning Objects

A method can return any type of data, including class types that you create.

Example:

```
class Test {
    int a;

    Test(int i) {
        a = i;
    }

    Test incrByTen() {
        Test temp = new Test(a + 10);
        return temp;
    }
}
```

[Handwritten Notes Page 42]

```
class RetOb {
    public static void main(String args[]) {
        Test ob1 = new Test(2);
        Test ob2; // Reference variable

        ob2 = ob1.incrByTen();
        System.out.println("ob1.a: " + ob1.a);
        System.out.println("ob2.a: " + ob2.a);

        ob2 = ob2.incrByTen();
        System.out.println("ob2.a after second increase: " + ob2.a);
    }
}
```

Output:

```
ob1.a: 2
ob2.a: 12
ob2.a after second increase: 22
```

Variable Types:

- - **Primitive Variable:** The variables that hold actual primitive values (e.g., int, float).
- - **Reference Variable:** The variable that stores the reference (memory address) pointing to an object.

Program Flow & Explanation:

Here, the program starts execution from the **main** method:

1. **Test ob1 = new Test(2);**: **ob1** of the class **Test** is created. At that time, the **Test(int i)** constructor is invoked with argument **2**, initializing **ob1.a = 2**.
2. **Test ob2;**: A reference variable **ob2** is declared; no object memory has been allocated yet (it is **null**).
3. **ob2 = ob1.incrByTen();**: The **incrByTen()** method is called on **ob1** and its return value is assigned to **ob2**.
4. Inside **incrByTen()**, **a** has value **2**. The expression **a + 10** evaluates to **12**, and **new Test(12)** is instantiated.
5. The constructor **Test(int i)** assigns **12** to **a**, and the newly created object reference **temp** is returned.
6. In this way, **ob2.a** points to the new object and prints **12** as output. Calling **ob2.incrByTen()** again creates another object with **a = 22** and updates **ob2**.

[[Handwritten Notes Page 43](#)]

INHERITANCE BASICS

Inheritance is one of the key features of Object-Oriented Programming (OOP) that allows us to create a new class from an existing class.

- - The new class that is created is known as the **Subclass** (child class / derived class).
- - The existing class from which the child class is derived is known as the **Superclass** (parent class / base class).

> **Note:** To inherit a class, you simply incorporate the definition of one class into another by using the **extends** keyword.

>

> **Inherit meaning:** Allowing all accessible methods and variables from one class to be accessible and reused by another class.

Example:

```
class A {
    int i, j;

    void showij() {
        System.out.println("i and j: " + i + " " + j);
    }
}

// Create a subclass by extending class A
class B extends A {
    int k;

    void showk() {
        System.out.println("k: " + k);
    }

    void sum() {
        System.out.println("i + j + k: " + (i + j + k));
    }
}

class SimpleInheritance {
    public static void main(String args[]) {
        A superOb = new A();
        B subOb = new B();
    }
}
```

[[Handwritten Notes Page 44](#)]

```
// The superclass may be used by itself
superOb.i = 10;
superOb.j = 20;
System.out.println("Contents of superOb: ");
superOb.showij();
System.out.println();

/* The subclass has access to all public/package members
   of its superclass. */
subOb.i = 7;
subOb.j = 8;
subOb.k = 9;
System.out.println("Contents of subOb: ");
subOb.showij();
subOb.showk();
System.out.println();

System.out.println("Sum of i, j and k in subOb:");
subOb.sum();
}
}
```

Output:

Contents of superOb:
i and j: 10 20

Contents of subOb:
i and j: 7 8
k: 9

Sum of i, j and k in subOb:
i + j + k: 24

NOTE:

- - The subclass includes all accessible members of the superclass.
- - The superclass knows only about its own members; it does not have knowledge of the subclass members.
- - A subclass object can access the members of the superclass. Whereas a superclass object **cannot** access the members of the subclass.

[Handwritten Notes Page 45]

- - A class member that has been declared as **private** in a superclass cannot be accessed directly by a subclass object.

The `super` Keyword

A subclass can call a constructor defined by its superclass by use of the following form of **super**:

Syntax:

```
super(parameter_list);
```

Here, **parameter_list** specifies any parameters needed by the constructor in the superclass.

- - **super()** must always be the first statement executed inside a subclass constructor.

Example: The Complete Implementation of `BoxWeight`

```
class Box {
    private double width;
    private double height;
    private double depth;

    // Pass object to constructor (clone constructor)
    Box(Box ob) {
        width = ob.width;
        height = ob.height;
        depth = ob.depth;
    }

    // Constructor used when all dimensions specified
    Box(double w, double h, double d) {
        width = w;
        height = h;
        depth = d;
    }
}
```

[[Handwritten Notes Page 46](#)]

```
// Constructor used when no dimensions specified
Box() {
    width = -1;
    height = -1;
    depth = -1;
}

// Constructor used when cube is created
Box(double len) {
    width = height = depth = len;
}

// Compute and return volume
double volume() {
    return width * height * depth;
}
}

class BoxWeight extends Box {
    double weight;

    // Constructor clone of an object
    BoxWeight(BoxWeight ob) {
        super(ob); // NOTE: super is called with an object of type BoxWeight, which matches Box(Bo
x ob)
        weight = ob.weight;
    }

    // Constructor when all parameters are specified
    BoxWeight(double w, double h, double d, double m) {
        super(w, h, d); // Call superclass constructor
        weight = m;
    }

    // Default constructor
    BoxWeight() {
        super();
        weight = -1;
    }
}
```

[Handwritten Notes Page 47]

```
// Constructor used when cube is created
BoxWeight(double len, double m) {
    super(len);
    weight = m;
}

class DemoSuper {
    public static void main(String args[]) {
        BoxWeight mybox1 = new BoxWeight(10, 20, 15, 34.3);
        BoxWeight mybox2 = new BoxWeight(2, 3, 4, 0.076);
        BoxWeight mybox3 = new BoxWeight();
        BoxWeight mycube = new BoxWeight(3, 2);
        BoxWeight myclone = new BoxWeight(mybox1);
        double vol;

        vol = mybox1.volume();
        System.out.println("Volume of mybox1 is " + vol);
        System.out.println("Weight of mybox1 is " + mybox1.weight);

        vol = mybox2.volume();
        System.out.println("Volume of mybox2 is " + vol);
        System.out.println("Weight of mybox2 is " + mybox2.weight);

        vol = mybox3.volume();
        System.out.println("Volume of mybox3 is " + vol);
        System.out.println("Weight of mybox3 is " + mybox3.weight);

        vol = myclone.volume();
        System.out.println("Volume of myclone is " + vol);
        System.out.println("Weight of myclone is " + myclone.weight);
        System.out.println();
    }
}
```

[Handwritten Notes Page 48]

```
        vol = mycube.volume();
        System.out.println("Volume of mycube is " + vol);
        System.out.println("Weight of mycube is " + mycube.weight);
    }
}
```

Output:

```
Volume of mybox1 is 3000.0
Weight of mybox1 is 34.3
Volume of mybox2 is 24.0
Weight of mybox2 is 0.076
Volume of mybox3 is -1.0
Weight of mybox3 is -1.0
Volume of myclone is 3000.0
Weight of myclone is 34.3
Volume of mycube is 27.0
Weight of mycube is 2.0
```

Second Use for `super`

super can also be used to access a member of the superclass that has been hidden by a member of the subclass.

Example:

```
class A {
    int i;
}

class B extends A {
    int i; // This i hides the i in A

    B(int a, int b) {
        super.i = a; // refers to i in superclass A
        i = b;       // refers to i in subclass B
    }

    void show() {
        System.out.println("i in superclass: " + super.i); // prints i in superclass
        System.out.println("i in subclass: " + i);          // prints i in subclass
    }
}
```

[Handwritten Notes Page 49]

Why We Need Inheritance?

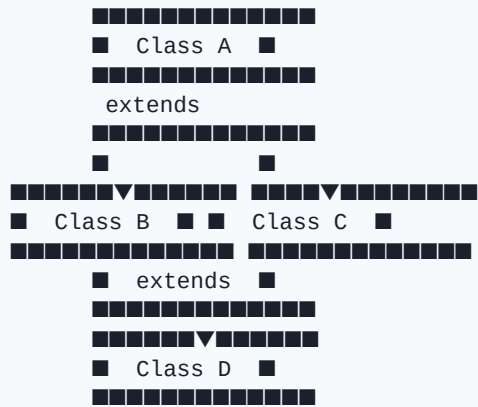
The most important use of inheritance in Java is **code reusability**. The code that is present in the parent class can be directly reused by the child class without rewriting it.

Types of Inheritance

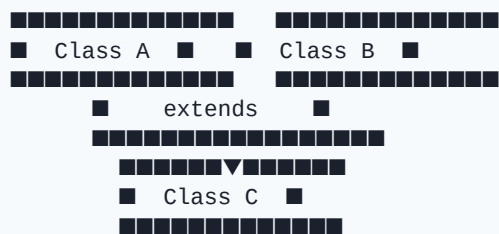
- **1. Single Inheritance:** In single inheritance, a single subclass extends from a single superclass.
- **2. Multilevel Inheritance:** In multilevel inheritance, a subclass extends from a superclass, and then that subclass acts as a superclass for another class.
- **3. Hierarchical Inheritance:** In hierarchical inheritance, multiple subclasses extend from a single superclass.
- **4. Multiple Inheritance:** In multiple inheritance, a single subclass extends from multiple superclasses. (*Note: Not supported in Java through classes; it is supported via interfaces.*)
- **5. Hybrid Inheritance:** Hybrid inheritance is a combination of two or more types of inheritance.

Inheritance Diagrams:

Hybrid Inheritance:



Multiple Inheritance:



(Class C attempting to extend both Class A and Class B)

[Handwritten Notes Page 50]

When Constructors Are Called

When a class hierarchy is created, in what order are the constructors for the classes that make up the hierarchy called?

For example, given a subclass called **B** and a superclass called **A**, is **A**'s constructor called before **B**'s or vice-versa? Let's look into it.

Example:

```
class A {
    A() {
        System.out.println("Inside A's constructor");
    }
}

class B extends A {
    B() {
        System.out.println("Inside B's constructor");
    }
}

class C extends B {
    C() {
        System.out.println("Inside C's constructor");
    }
}

class CallingCons {
    public static void main(String args[]) {
        C c = new C();
    }
}
```

Output:

```
Inside A's constructor
Inside B's constructor
Inside C's constructor
```

[\[Handwritten Notes Page 51 \]](#)

From this, we can conclude that constructors are called in order of derivation, from superclass down to subclass:

- - When object **C** is created, first **A**'s constructor, then **B**'s, and then **C**'s constructor is called.
- - If **B** is instantiated, then **A**'s and then **B**'s constructor is called.

Method Overriding

If the same method is defined in both the superclass and the subclass (having the same name and parameter list), then the method of the subclass **overrides** the method of the superclass. This is known as **Method Overriding**.

Example:

```
class A {
    int i, j;

    A(int a, int b) {
        i = a;
        j = b;
    }

    void show() {
        System.out.println("i and j: " + i + " " + j);
    }
}

class B extends A {
    int k;

    B(int a, int b, int c) {
        super(a, b);
        k = c;
    }

    void show() {
        super.show(); // calls superclass show()
        System.out.println("k: " + k);
    }
}
```

[Handwritten Notes Page 52]

```
class Override {
    public static void main(String args[]) {
        B subOb = new B(1, 2, 3);
        subOb.show();
    }
}
```

Output:

```
i and j: 1 2
k: 3
```

NOTE:

- - Method overriding occurs **only** when the names and the type signatures (parameter lists) of the two methods are identical.
- - **Type Signature:** The method parameters. If the parameters are different, the method is **overloaded**, not overridden.

String Handling

The String Constructors

The **String** class supports several constructors. (**String** is an inbuilt class in **java.lang**).

1. Empty String:

To create an empty string, call the default constructor:

```
String s = new String();
```

\$\rightarrow\$ This will create an instance of **String** with no characters in it.

2. String initialized by an array of characters:

Syntax:

```
String(char chars[])
```

Example:

```
char chars[] = { 'a', 'b', 'c' };  
String s = new String(chars);
```

All characters from the array are converted into the String and stored in **s**.

[Handwritten Notes Page 53]

3. Subrange of Character Array

To create a string from a subrange of a character array:

Syntax:

```
String(char chars[], int startIndex, int numChars)
```

- - **chars[]**: Array that contains the characters.
- - **startIndex**: The index at which the subrange begins.
- - **numChars**: Number of characters to use from the subrange.

Example:

```
char chars[] = { 'a', 'b', 'c', 'd', 'e', 'f' };  
// Index:      0    1    2    3    4    5  
  
String s = new String(chars, 2, 3);  
// This initializes s with "cde"  
// i.e., from index 2, it takes 3 characters.
```

NOTE:

You can construct a **String** object that contains the same character sequence as another **String** object using this constructor:

```
String(String strObj)
```

Example:

```
char c[] = { 'J', 'a', 'v', 'a' };
String s1 = new String(c);
String s2 = new String(s1);

System.out.println(s1);
System.out.println(s2);
```

Output:

```
Java
Java
```

4. Convert Array of Bytes into String (Byte Array to String)

Example:

```
class SubstringCons {
    public static void main(String args[]) {
        byte ascii[] = { 65, 66, 67, 68, 69, 70 };

        String s1 = new String(ascii); // Array of ASCII numbers converted into letters ("ABCDEF")
        System.out.println(s1);

        String s2 = new String(ascii, 2, 3); // Extracts 3 characters starting from index 2 ("CDE")
    }
    System.out.println(s2);
}
```

[\[Handwritten Notes Page 54 \]](#)

String Length

```
char chars[] = { 'a', 'b', 'c' };
String s = new String(chars);
int length = s.length(); // To find the string length
```

You can also find the length of a string literal directly:

```
String s2 = "abc";
System.out.println("abc".length());
```

String Concatenation

```
String age = "9";
String s = "He is " + age + " years old.";
```

We use the **+** operator to concatenate strings.

```
System.out.println(s);
```

Output:

```
He is 9 years old.
```

String Concatenation with Other Data Types

Example 1:

```
int age = 9;
String s = "He is " + age + " years old.";
```

Example 2:

```
int num = 4;
int i = 2;
System.out.println(num + " x " + i + " = " + (num * i));
// Output: 4 x 2 = 8
```

To Convert an Integer to a String

- 1. Using **Integer.toString()**:

```
int number = 123;
String str = Integer.toString(number);
```

- 2. Using **String.valueOf()**:

```
int number = 123;
String str = String.valueOf(number);
```

[Handwritten Notes Page 55]

- 3. Using **String Concatenation**:

```
int number = 123;
String str = "" + number;
```

- 4. Using **String.format()**:

```
int number = 123;
String str = String.format("%d", number);
```

Character Extraction

`charAt()`

To extract a single character from a String.

Syntax:

```
char charAt(int where) // 'where' specifies the index
```

Example:

```
char ch;  
ch = "abc".charAt(1); // assigns 'b' to ch
```

`getChars()`

Used to extract more than one character at a time.

(Note: [getBytes\(\)](#) can also be used to extract into a byte array).

Syntax:

```
void getChars(int sourceStart, int sourceEnd, char target[], int targetStart)
```

- - **sourceStart**: Specifies the index of the beginning of the substring.
- - **sourceEnd**: Specifies the index after the end of the substring (exclusive).
- - **target[]**: The array that will receive the characters.
- - **targetStart**: The index within **target** at which copying begins.

[Handwritten Notes Page 56]

(Continuation of [getChars](#) explanation)

- - **targetStart**: The index within **target** at which the substring will be copied.

Example:

```
class getCharsDemo {  
    public static void main(String args[]) {  
        String s = "This is a demo of the getChars method.";  
        int start = 10;  
        int end = 14;  
        char buf[] = new char[end - start];  
  
        s.getChars(start, end, buf, 0);  
        System.out.println(buf);  
    }  
}
```

Output:

```
demo
```

`toCharArray()`

If you want to convert all the characters in a **String** object into a character array, simply call **toCharArray()**:

Example:

```
String s = "GeeksForGeeks";  
char[] gfg = s.toCharArray();  
System.out.println(gfg);
```

String Comparison

(Introduced on next page)

[[Handwritten Notes Page 57](#)]

String Comparison

`equals()`

It returns **true** if all the characters in both strings are the same, else **false**. It is case-sensitive.

Example:

```
String s1 = "Hello";
String s2 = "Hello";
String s3 = "HELLO";

System.out.println(s1.equals(s2)); // Output: true
System.out.println(s1.equals(s3)); // Output: false
```

`equalsIgnoreCase()`

Syntax:

```
boolean equalsIgnoreCase(String str)
```

It compares two strings without considering letter case (i.e. treats uppercase and lowercase characters as equal).

Example:

```
System.out.println(s1.equalsIgnoreCase(s3)); // Output: true
```

`regionMatches()`

The `regionMatches()` method compares a specific region inside a string with another specific region in another string.

Syntax:

```
boolean regionMatches(int startIndex, String str2, int str2StartIndex, int numChars)
// Overload with ignoreCase:
boolean regionMatches(boolean ignoreCase, int startIndex, String str2, int str2StartIndex, int numChars)
```

Example:

```
String str1 = "Hello World";
String str2 = "World";

boolean match = str1.regionMatches(true, 6, str2, 0, 5);
System.out.println(match);
```

Output:

```
true
```

*(Returns **true** because index 6 of **str1** is **"World"**, matching **str2** for 5 characters, case-insensitively).*

[[Handwritten Notes Page 58](#)]

`startsWith()` and `endsWith()`

- - **startsWith()** determines whether a given **String** begins with a specified string.
- - **endsWith()** determines whether the **String** in question ends with a specified string.

Example:

```
"Foobar".endsWith("bar"); // returns true
"Foobar".startsWith("Foo"); // returns true
```

Also supports an offset index:

```
"Foobar".startsWith("bar", 3); // returns true
//                               ▲
//                               index
```

Here **index** specifies the index into the invoking string at which point the search begins.

`equals()` versus `==`

1. It is important to understand that the **equals()** method and the **==** operator perform two different operations:
2. The **equals()** method compares the characters inside a **String** object (content/value equality).
3. The **==** operator compares two object references to see whether they refer to the exact same instance in memory (reference equality).

Example:

```
String s1 = "Hello";
String s2 = new String(s1);

System.out.println(s1.equals(s2)); // Output: true
System.out.println(s1 == s2);      // Output: false
```

Output:

```
true
false
```

[Handwritten Notes Page 59]

`compareTo()`

1. For sorting applications, you need to know whether a string is less than, equal to, or greater than another.
2. A string is less than another if it comes before the other in dictionary (lexicographical) order.
3. A string is greater than another if it comes after the other in dictionary order.

Syntax:

```
int compareTo(String str)
```

| Value | Meaning |

| :--- | :--- |

| **Less than zero (< 0)** | The invoking string is less than **str**. |

| **Greater than zero (> 0)** | The invoking string is greater than **str**. |

| **Zero (0)** | The two strings are equal. |

Code to Arrange Words in Dictionary Order using `compareTo`:

```
class SortString {
    static String arr[] = {
        "Now", "is", "the", "time", "for", "all",
        "good", "men", "to", "come", "to", "the", "aid", "of",
        "their", "country"
    };

    public static void main(String args[]) {
        for (int j = 0; j < arr.length; j++) {
            for (int i = j + 1; i < arr.length; i++) {
                if (arr[i].compareTo(arr[j]) < 0) {
                    String t = arr[j];
                    arr[j] = arr[i];
                    arr[i] = t;
                }
            }
            System.out.println(arr[j]);
        }
    }
}
```

Output:

```
Now
aid
all
come
country
for
good
is
men
of
the
the
their
time
to
to
```

> **Note:** The word **"Now"** comes out before all the others because it begins with an uppercase letter, whose ASCII value is less than lowercase letters.

[Handwritten Notes Page 60]

`compareToIgnoreCase()`

To ignore case differences when comparing strings:

```
int compareToIgnoreCase(String str)
```

Searching Strings

- 1. **indexOf()**: Searches for the first occurrence of a character or substring.
- 2. **lastIndexOf()**: Searches for the last occurrence of a character or substring.

Given string:

```
String s = "Now is the time for all good men to come to the aid of their country";
```

1. To search for the first occurrence of a character:

```
s.indexOf('t'); // Gives the index where the first 't' occurs
```

2. To search for the last occurrence of a character:

```
s.lastIndexOf('t'); // Gives the index where the last 't' occurs
```

3. To search for the first and last occurrence of a substring:

```
s.indexOf("the");      // Output = 7  
s.lastIndexOf("the");  // Output = 55
```

*(If the character or substring is not present, both methods return **-1**)*.

4. To search from a specific starting point:

```
s.indexOf('t', 10);      // Output = 11 (searches forward from index 10)  
s.lastIndexOf('t', 60); // Output = 55 (searches backward from index 60 towards 0)  
  
s.indexOf("the", 10);    // Searches forward for "the" starting from index 10  
s.lastIndexOf("the", 60); // Searches backward for "the" starting from index 60
```

[Handwritten Notes Page 61]

Modifying a String

`substring()`

Extracts a substring from the given string.

```
String org = "This is a test.";
String result = org.substring(0, 4);
```

> **Note:** The ending index is exclusive (it won't consider the 4th index). It extracts characters from index 0 to 3.

Output:

```
This
```

Example 2:

```
String result = org.substring(2);
```

Prints from the 2nd index to the end of the string.

```
---
```

`concat()`

Concatenates the specified string to the end of this string.

```
String s1 = "one";
String s2 = s1.concat("two");
```

Output:

```
onetwo
```

```
---
```

`replace()`

Replaces occurrences of a character with another character.

```
String s = "Hello".replace('l', 'w');
```

```
---
```

`trim()`

The **trim()** method returns a copy of the invoking string from which any leading and trailing whitespace has been removed.

Example:

```
String s = " Hello World ".trim();
```

Output:

```
Hello World
```

It is useful when user input contains leading or trailing whitespaces; `trim()` removes them.

[Handwritten Notes Page 62]

```
// Example: Reading input and removing surrounding whitespace
str = br.readLine();
str = str.trim(); // removes whitespaces
```

Changing the Case of Characters within a String

```
String s = "This is a test.";
String upper = s.toUpperCase(); // Note: 'U' and 'C' are capital in toUpperCase()
String lower = s.toLowerCase();
```

Example: Convert First Letter to Uppercase

```
System.out.println(A.substring(0, 1).toUpperCase() + A.substring(1));
```

Joining Strings

The `String.join()` method joins strings with a specified delimiter.

```
String result = String.join(" ", "Alpha", "Beta", "Gamma");
```

Output:

```
Alpha Beta Gamma
```

Note: `Integer.toBinaryString()`

```
Integer.toBinaryString(num1);
```

Converts an integer to its binary string representation.

[Handwritten Notes Page 63]

Java Abstract Class and Abstract Methods

- 1. An abstract class in Java cannot be instantiated (i.e., we cannot create objects of abstract classes directly).

- 2. We use the **abstract** keyword to declare an abstract class.
- 3. An abstract class can have both regular (concrete) methods and abstract methods.

Example:

```
abstract class Language {
    abstract void method1();

    void method2() {
        System.out.println("This is regular method");
    }
}
```

Java Abstract Method

- 1. A method that doesn't have a body is known as an abstract method.
- 2. We use the **abstract** keyword to declare abstract methods.

Example:

```
abstract void display();
```

Here, **display()** is an abstract method. The body of **display()** is replaced with a semicolon (;).

> NOTE:

> If a class contains an abstract method, the class **must** be declared abstract; otherwise, it will throw a compile-time error.

```
class Language {
    abstract void method1(); // Compile error: class Language must be declared abstract
}
```

[Handwritten Notes Page 64]

- 3. Abstract classes cannot be instantiated, but we can create subclasses from them. We can then access members of the abstract class using an object of the subclass.
- 4. We can inherit from an abstract class using the **extends** keyword.
- 5. Abstract methods must be defined/implemented in subclasses.
- 6. The abstract methods can be implemented across the class hierarchy by concrete subclasses.

Example (Case 1):

```
abstract class Hello {
    abstract void printing();
}

class Hi extends Hello {
    void printing() {
        System.out.println("Virendra");
    }
}

class Practice {
    public static void main(String[] args) {
        Hi obj = new Hi();
        obj.printing();
    }
}
```

[\[Handwritten Notes Page 65 \]](#)

Example (Case 2):

```
abstract class Animal {
    abstract void makeSound();

    public void eat() {
        System.out.println("I can eat");
    }
}

class Dog extends Animal {
    public void makeSound() {
        System.out.println("Bark bark");
    }
}

class Main {
    public static void main(String[] args) {
        Dog d1 = new Dog();
        d1.makeSound();
        d1.eat();
    }
}
```

> NOTE:

> If the **Dog** class does not provide an implementation for the abstract method **makeSound()**, then **Dog** must also be declared **abstract**. This is because the subclass **Dog** inherits the abstract method **makeSound()** from **Animal**.

>

> *(Inheritance allows all accessible methods and variables from one class to be inherited by another class).*

[\[Handwritten Notes Page 66 \]](#)

Access Constructor of Abstract Classes

An abstract class can have constructors just like a regular class. We can access the constructor of an abstract class from a subclass using the **super** keyword.

Example:

```
abstract class Animal {
    Animal() {
        // Abstract class constructor
    }
}

class Dog extends Animal {
    Dog() {
        super(); // Calls superclass constructor
    }
}
```

JAVA INTERFACE

- 1. An interface is a fully abstract contract in Java. It contains a collection of abstract methods.
- 2. We use the **interface** keyword to create an interface in Java.

Example:

```
interface Language {
    public void getType();
    public void getVersion();
}
```

Methods declared in an interface are implicitly **public** and **abstract**, even though the **abstract** modifier is omitted.

> **NOTE:**

> An empty interface (an interface without any members) is called a **Tag Interface** or **Marker Interface** (e.g., **Serializable**, **Cloneable**).

[[Handwritten Notes Page 67](#)]

Accessing Implementations Through Interface Reference

Example:

```

interface Callback {
    void callback(int param);
}

class Client implements Callback {
    public void callback(int p) {
        System.out.println("callback called with " + p);
    }

    void nonIfaceMeth() {
        System.out.println("Hello");
    }
}

class AnotherClient implements Callback {
    public void callback(int p) {
        System.out.println("Another version of callback");
        System.out.println("p sq value is " + (p * p));
    }
}

class Practice {
    public static void main(String args[]) {
        Callback c = new Client();
        // 'c' can access only the callback method defined in Callback interface.
        // It cannot access nonIfaceMeth() via interface reference.

        AnotherClient ob = new AnotherClient();
        // 'ob' can access all methods in AnotherClient.

        c.callback(42); // Calls callback method in Client class
    }
}

```

[Handwritten Notes Page 68]

```

        ob.callback(200); // Calls callback method of AnotherClient
        // ob.nonIfaceMeth(); -> Invalid: AnotherClient does not have nonIfaceMeth()
        // We cannot access a method belonging to Client class using an object of AnotherClient.
    }
}

```

> NOTE:

> If a class implements an interface but does not fully implement all the methods required by that interface, then that class **must** be declared as **abstract**.

Example:

```
abstract class Incomplete implements Callback {
    int a, b;

    void show() {
        System.out.println(a + " " + b);
    }
}
```

- - Here, the class **Incomplete** does not implement **callback()** and therefore must be declared as **abstract**.
- - Any class that inherits **Incomplete** must implement **callback()** or be declared **abstract** itself.

[\[Handwritten Notes Page 69 \]](#)

NESTED INTERFACE

An interface can be declared as a member of a class or another interface. Such an interface is called a **member interface** or a **nested interface**.

Example:

```
class A {
    // Nested interface
    public interface NestedIF {
        boolean isNotNegative(int x);
    }
}

// B implements the nested interface
class B implements A.NestedIF {
    public boolean isNotNegative(int x) {
        return x < 0 ? false : true;
    }
}

class NestedIFDemo {
    public static void main(String args[]) {
        // Use a nested interface reference:
        A.NestedIF nif = new B();

        if (nif.isNotNegative(10)) {
            System.out.println("10 is not negative");
        }
        if (nif.isNotNegative(-12)) {
            System.out.println("this won't be displayed");
        }
    }
}
```

Interface Can Be Extended

One interface can inherit another interface using the keyword **extends**.

Example:

```
interface A {  
    void meth1();  
    void meth2();  
}
```

[\[Handwritten Notes Page 70 \]](#)

```
// Interface B extends interface A  
interface B extends A {  
    void meth3();  
}  
  
// MyClass must implement all methods from both A and B  
class MyClass implements B {  
    public void meth1() {  
        System.out.println("Implement meth1().");  
    }  
  
    public void meth2() {  
        System.out.println("Implement meth2().");  
    }  
  
    public void meth3() {  
        System.out.println("Implement meth3().");  
    }  
}  
  
class IFExtend {  
    public static void main(String arg[]) {  
        MyClass ob = new MyClass();  
        ob.meth1();  
        ob.meth2();  
        ob.meth3();  
    }  
}
```

Default Methods in Java Interfaces

We can add methods with implementations inside an interface. These methods are called **default methods** (introduced in Java 8).

To declare default methods inside interfaces, we use the **default** keyword.

[\[Handwritten Notes Page 71 \]](#)

Why Default Methods?

Let's take a scenario to understand why default methods were introduced in Java:

- - Suppose we need to add a new method to an existing interface.
- - If we add an abstract method to an interface, all classes that implement that interface must provide an implementation for that method.
- - If a large number of classes were implementing this interface, we need to track all these classes and modify each one.
- - Default methods allow adding new functionality to interfaces without breaking existing implementing classes.

Example:

```
interface Polygon {
    void getArea();

    default void getSides() {
        System.out.println("I can get sides of a polygon.");
    }
}

class Rectangle implements Polygon {
    public void getArea() {
        int length = 6;
        int breadth = 5;
        int area = length * breadth;
        System.out.println("The area of the rectangle is " + area);
    }

    // Overriding the default method
    public void getSides() {
        System.out.println("I have 4 sides.");
    }
}

class Square implements Polygon {
    public void getArea() {
        int length = 5;
        int area = length * length;
    }
}
```

[\[Handwritten Notes Page 72 \]](#)

```
        System.out.println("The area of the square is " + area);
    }
}

class Main {
    public static void main(String[] args) {
        Rectangle r1 = new Rectangle();
        r1.getArea();
        r1.getSides();

        Square s1 = new Square();
        s1.getArea();
        s1.getSides();
    }
}
```

Explanation:

- - Here, we have created two classes, **Rectangle** and **Square**, that implement **Polygon**.
- - The **Rectangle** class provides an implementation of **getArea()** and overrides the **getSides()** method.
- - The **Square** class only provides an implementation of **getArea()**.
- - When calling **getSides()** on **Rectangle**, the overridden method is executed.
- - In the case of **Square**, the **default** method from **Polygon** is executed.

Use Static Methods in an Interface

Interfaces can also contain **static** methods with implementations (since Java 8):

```
public interface MyIF {
    int getNumber();

    default String getString() {
        return "Default String";
    }

    static int getDefaultNumber() {
        return 0;
    }
}
```

[Handwritten Notes Page 73]

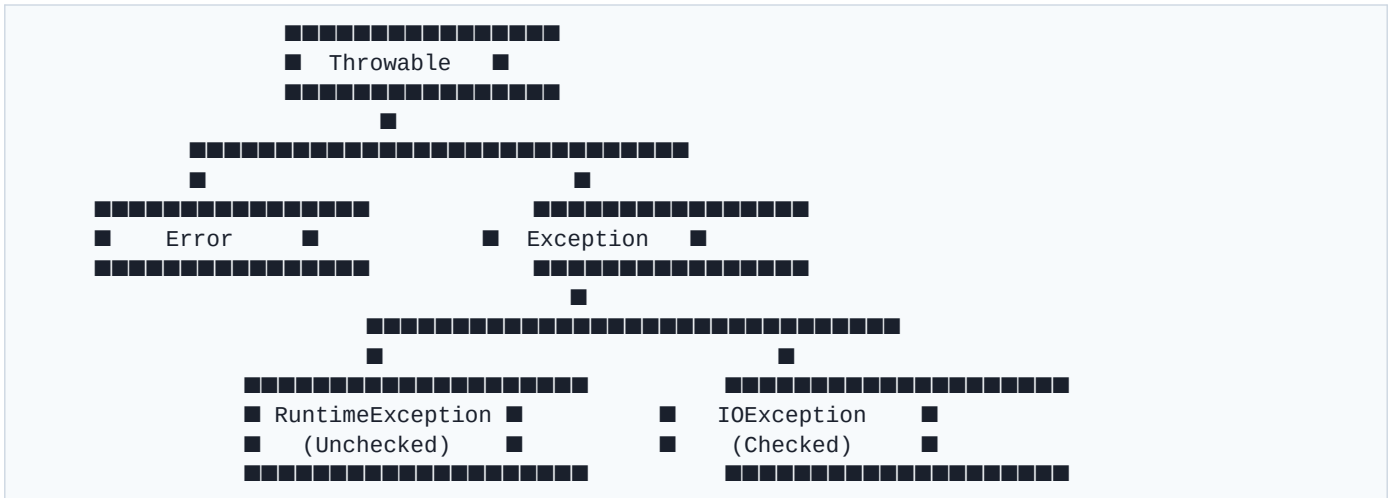
The **getDefaultNumber()** static method can be called directly via the interface name:

```
int defNum = MyIF.getDefaultNumber();
```

JAVA EXCEPTIONS

- 1. An exception is an unexpected event that occurs during program execution.
- 2. It affects the flow of program instructions, which can cause the program to terminate abnormally.

Java Exception Hierarchy



Throwable class is the root class in the exception hierarchy.

Errors

- - Errors represent irrecoverable conditions such as Java Virtual Machine running out of memory (**OutOfMemoryError**), memory leaks, stack overflow errors (**StackOverflowError**), library incompatibility, etc.
- - Errors are usually beyond the control of the programmer, and we should not try to handle them.

[Handwritten Notes Page 74]

Exceptions

- - Exceptions can be caught and handled by the programmer.
- - When an exception occurs within a method, the method creates an object called the **exception object**.
- - The exception object contains information about the exception, such as its name, description, and state of the program when it occurred.

1. Runtime Exception (Unchecked Exception)

- - A runtime exception happens due to programming flaws or logical errors.
- - They are also known as **unchecked exceptions**.

- - These exceptions are not checked at compile-time, but are checked at runtime.

Examples:

- - Improper use of an API $\rightarrow$ `IllegalArgumentException`
- - Dividing a number by 0 $\rightarrow$ `ArithmeticException`

2. IO Exception (Checked Exception)

- - An `IOException` is a **checked exception**.
- - Checked exceptions are checked by the compiler at compile-time, and the programmer is required/prompted to handle these exceptions.

Example:

- - Trying to open a file that doesn't exist results in `FileNotFoundException`

[[Handwritten Notes Page 75](#)]

try...catch

The `try...catch` block in Java is used to handle exceptions and prevents the abnormal termination of the program.

- - The `try` block includes the code that might generate an exception.
- - The `catch` block includes the code that is executed when an exception occurs inside the `try` block.

Example:

```
class Exc2 {
    public static void main(String args[]) {
        int d, a;
        try {
            d = 0;
            a = 42 / d; // Exception is thrown here
            System.out.println("This will not be printed.");
        } catch (ArithmeticException e) {
            System.out.println("Division by Zero.");
        }
        System.out.println("After catch statement.");
    }
}
```

MULTIPLE Catch Clauses

In some cases, more than one exception could be raised by a single piece of code. To handle this type of situation, you can specify two or more `catch` statements.

[\[Handwritten Notes Page 76 \]](#)

```
class MultipleCatches {
    public static void main(String args[]) {
        try {
            int a = args.length;
            System.out.println("a = " + a);
            int b = 42 / a;
            int c[] = { 1 };
            c[42] = 99;
        } catch (ArithmeticException e) {
            System.out.println("Divide by 0: " + e);
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Array index oob: " + e);
        }
        System.out.println("After try/catch blocks.");
    }
}
```

Explanation:

- - This program will cause a division-by-zero exception if it is started with no command-line arguments, since **a** will equal zero.
- - It will survive the division if you provide a command-line argument, setting **a** to something larger than zero. But it will cause an **ArrayIndexOutOfBoundsException** on array **c**.

Output (No Arguments):

```
C:\> java MultipleCatches
a = 0
Divide by 0: java.lang.ArithmeticException: / by zero
After try/catch blocks.
```

[\[Handwritten Notes Page 77 \]](#)**Output (With Arguments):**

```
C:\> java MultipleCatches TestArg
a = 1
Array index oob: java.lang.ArrayIndexOutOfBoundsException: Index 42 out of bounds for length 1
After try/catch blocks.
```

> NOTE:

> While using multiple catch statements, it is important to remember that exception subclasses must come before any of their superclasses.

> This is because a catch statement that uses a superclass will catch exceptions of that type plus any of its subclasses. Thus, a subclass catch block would never be reached if it came after its superclass.

Example:

```
class SuperSubCatch {
    public static void main(String args[]) {
        try {
            int a = 0;
            int b = 42 / a;
        } catch (Exception e) {
            // Superclass catch block
            System.out.println("Generic Exception catch.");
        }
        /* This catch is never reached because
        ArithmeticException is a subclass of Exception. */
        catch (ArithmeticException e) { // Compile-time ERROR: Unreachable code!
            System.out.println("This is never reached");
        }
    }
}
```

NESTED try STATEMENTS

The **try** statement can be nested. That is, a **try** statement can be inside the block of another **try**.

[[Handwritten Notes Page 78](#)]

```
class NestTry {
    public static void main(String args[]) {
        try {
            int a = args.length;
            int b = 42 / a;
            System.out.println("a = " + a);
            try { // Nested try block
                if (a == 1) {
                    a = a / (a - a); // Division by zero
                }
                if (a == 2) {
                    int c[] = { 1 };
                    c[42] = 99; // Out-of-bounds
                }
            } catch (ArrayIndexOutOfBoundsException e) {
                System.out.println("Array index out-of-bounds: " + e);
            }
        } catch (ArithmeticException e) {
            System.out.println("Divide by 0: " + e);
        }
    }
}
```

Explanation:

- - When you execute the program with no command-line arguments, a divide-by-zero exception is generated at [42 / a](#):

```
C:\> java NestTry
Divide by 0: java.lang.ArithmeticException: / by zero
```

- - When you execute the program with 1 command-line argument, a divide-by-zero exception occurs within the nested **try** block (**a / (a - a)**). Since the inner block only catches **ArrayIndexOutOfBoundsException**, the exception is passed to the outer **try** block where it is handled:

```
a = 1
Divide by 0: java.lang.ArithmeticException: / by zero
```

[Handwritten Notes Page 79]

- - If you execute with 2 command line arguments, the following output is produced:

```
a = 2
Array index out-of-bounds: java.lang.ArrayIndexOutOfBoundsException: Index 42 out of bounds for length 1
```

throw

It is possible for your program to throw an exception explicitly using the **throw** statement.

Syntax:

```
throw ThrowableInstance;
```

Here, **ThrowableInstance** must be an object of type **Throwable** or a subclass of **Throwable**. Non-Throwable classes, such as **String** and **Object**, cannot be used as exceptions.

> Note:

> The flow of execution stops immediately after the **throw** statement; any subsequent statements are not executed.

Example:

```
class ThrowDemo {
    static void demoproc() {
        try {
            throw new NullPointerException("demo");
        } catch (NullPointerException e) {
            System.out.println("Caught inside demoproc.");
            throw e; // Rethrow the exception back to main function
        }
    }

    public static void main(String args[]) {
        try {
            demoproc();
        } catch (NullPointerException e) {
            System.out.println("Recaught: " + e);
        }
    }
}
```

[Handwritten Notes Page 80]

Output:

```
Caught inside demoproc.
Recaught: java.lang.NullPointerException: demo
```

How to Create Standard Exception Objects:

```
throw new NullPointerException("demo");
```

throws

A **throws** clause lists the types of exceptions that a method might throw.

General Form of a Method Declaration with **throws**:

```
type method-name(parameter-list) throws exception-list {
    // body of method
}
```

Here, **exception-list** is a comma-separated list of exceptions that a method can throw.

Example (Checked Exception):

```
class ThrowsDemo {
    static void throwOne() throws IllegalAccessException {
        System.out.println("Inside throwOne.");
        throw new IllegalAccessException("demo");
    }

    public static void main(String args[]) {
        try {
            throwOne();
        } catch (IllegalAccessException e) {
            System.out.println("Caught: " + e);
        }
    }
}
```

[\[Handwritten Notes Page 81 \]](#)

Output (from Previous Example)

```
inside throwOne
Caught java.lang.IllegalAccessException: demo
```

Creating Your Own Exception Subclasses

Although Java's built-in exceptions handle most common errors, you will probably want to create your own exception types to handle situations specific to your applications.

The following example declares a new subclass of **Exception**:

```
// Example: Creating a custom exception class
class MyException extends Exception {
    private int detail;

    MyException(int a) {
        detail = a;
    }

    @Override
    public String toString() {
        return "MyException[" + detail + "]";
    }
}

class ExceptionDemo {
    static void compute(int a) throws MyException {
        System.out.println("Called compute(" + a + ")");
        if (a > 10) {
            throw new MyException(a);
        }
        System.out.println("Normal exit");
    }

    public static void main(String args[]) {
        try {
            compute(1);
            compute(20);
        } catch (MyException e) {
            // Continued on next page...
        }
    }
}
```

(Code continues on Page 82)

[Handwritten Notes Page 82]

(Continued from Page 81)

```
        } catch (MyException e) {
            System.out.println("Caught " + e);
        }
    }
}
```

Output:

```
Called compute(1)
Normal exit
Called compute(20)
Caught MyException[20]
```

Example 2: Custom Exception with Validation

```

class InvalidAgeException extends Exception {
    public InvalidAgeException(String message) {
        super(message);
    }
}

public class Main {
    public static void main(String[] args) {
        if (args.length == 0) {
            System.out.print("Please provide age as a command line argument");
            return;
        }

        try {
            int age = Integer.parseInt(args[0]);
            if (age < 18) {
                throw new InvalidAgeException("Age is less than 18");
            }
            System.out.println("Age is valid");
        } catch (InvalidAgeException e) {
            System.out.println(e.getMessage());
        } catch (NumberFormatException e) {
            System.out.println("Invalid number format: " + e.getMessage());
        }
    }
}

```

[\[Handwritten Notes Page 83 \]](#)

Enumerations

In Java, an **enum** is a type that has a fixed set of constant values.

enum is short for enumeration. We use the **enum** keyword to declare enums.

```

enum Size {
    SMALL, MEDIUM, LARGE, EXTRALARGE
}

```

NOTE:

- - The enum constants are usually represented in uppercase, but it is not mandatory.
- - The identifiers **SMALL**, **MEDIUM**, **LARGE**, etc., are called **enumeration constants**.
- - Each is implicitly declared as a **public static final** member of **Size**.

```

enum Apple {
    Jonathan, GoldenDel, RedDel, Winesap, Cortland
}

```

- - Once you have defined an enumeration, you can create a variable of that type.
- - However, even though enumerations define a class type, you **do not instantiate an enum using new**.

```
Apple ap;
```

Here, `ap` is of type `Apple`. The only values that can be assigned to it are those defined by the enumeration:

```
ap = Apple.RedDel;
```

[Handwritten Notes Page 84]

- - Two enumeration constants can be compared for equality by using the `==` relational operator:

```
if (ap == Apple.GoldenDel)
```

- - An enumeration value can also be used to control a `switch` statement.

Example: Using Enum with `switch`

```
enum Apple {
    Jonathan, GoldenDel, RedDel, Winesap, Cortland
}

class EnumDemo {
    public static void main(String args[]) {
        Apple ap;
        ap = Apple.RedDel;
        System.out.println("Value of ap: " + ap);
        System.out.println();

        ap = Apple.GoldenDel;

        // Compare two enum values
        if (ap == Apple.GoldenDel) {
            System.out.println("ap contains GoldenDel.");
        }

        // Use an enum to control a switch statement
        switch (ap) {
            case Jonathan:
                System.out.println("Jonathan is red.");
                break;
            case GoldenDel:
                System.out.println("Golden Delicious is yellow.");
                break;
            case RedDel:
                System.out.println("Red Delicious is red.");
                break;
        }
    }
}
```

(Output continues on Page 85)

[Handwritten Notes Page 85]

(Continued from Page 84)

Output:

Value of ap: RedDel

ap contains GoldenDel.
Golden Delicious is yellow.

The `values()` and `valueOf()` Methods

1. Enum `values()`

The **values()** method returns an array of the enum type containing all the enum constants.

```
enum Apple {
    Jonathan, GoldenDel, RedDel, Winesap, Cortland
}

class EnumDemo2 {
    public static void main(String args[]) {
        Apple ap;
        Apple allapples[] = Apple.values();

        // Here all the enum constants will be stored in the allapples[] array.
        for (Apple a : allapples) { // Prints the elements in the array
            System.out.println(a);
        }
    }
}
```

2. Enum `valueOf()`

The **valueOf()** method takes a **String** and returns an enum constant having the same String name.

(Continued on Page 86)

[\[Handwritten Notes Page 86 \]](#)

*(Continued from Page 85 - Enum **valueOf()**)*

```
enum Color {
    RED, GREEN, BLUE
}

Color col = Color.valueOf("RED");
```

- - It returns the enum constant that matches the given String.
- - If no matching constant is found, it throws an **IllegalArgumentException**.
- - It is case-sensitive, so ensure that the String matches the constant's name exactly.

3. Java Enum `ordinal()`

The `ordinal()` method returns the position of an enum constant.

(ordinal meaning: index / position)

```
enum Size {
    SMALL, MEDIUM, LARGE, EXTRALARGE
    // 0      1      2      3
}

Size.SMALL.ordinal(); // returns 0
```

4. Enum `compareTo()`

The `compareTo()` method compares the enum constants based on their ordinal values.

```
enum Size {
    SMALL, MEDIUM, LARGE
}

public class EnumCompareToExample {
    public static void main(String[] args) {
        Size s1 = Size.SMALL;
        Size s2 = Size.MEDIUM;

        System.out.println(s1.compareTo(s2)); // -1 (SMALL comes before MEDIUM)
        System.out.println(s2.compareTo(s1)); // 1 (MEDIUM comes after SMALL)
        System.out.println(s1.compareTo(s1)); // 0 (Same ordinal)
    }
}
```

[\[Handwritten Notes Page 87 \]](#)

5. Enum `toString()`

The `toString()` method returns the String representation of the enum constants.

```
Size.SMALL.toString(); // returns "SMALL"
```

Java Enumerations Are Class Types

- 1. A Java enumeration is a class type.
- 2. Although you don't instantiate an enum using `new`, it otherwise has much the same capabilities as a class.
- 3. You can use constructors, add methods, add instance variables, and even implement interfaces.
- 4. An enumeration constant is an object of its enumeration type.
- 5. Thus, when you define a constructor for an enum, the constructor is called when each enumeration constant is created.

NOTE:

- 1. Enum constructors are **private** (accessible within the class) or package-private (accessible within the package). They cannot be declared **public** or **protected**.

Example: Enum Constructor

```
enum Size {
    SMALL("The size is small."),
    MEDIUM("The size is medium."),
    LARGE("The size is large.");

    private final String pizzaSize;
```

(Continued on Page 88)

[Handwritten Notes Page 88]

(Continued from Page 87)

```
    private Size(String pizzaSize) {
        this.pizzaSize = pizzaSize;
    }

    public String getSize() {
        return pizzaSize;
    }
}

class Main {
    public static void main(String[] args) {
        Size size = Size.SMALL; // Constructor call
        System.out.println(size.getSize());
    }
}
```

NOTE:

- 1. **Inheriting a class from an enum class:**

In Java, enum classes are **final** by default. Hence, we cannot inherit other classes from it (classes cannot extend an enum).

- 2. **Inheriting an enum class from another class:**

We cannot extend other classes in an enum class in Java.

This is because all enums in Java automatically inherit from **java.lang.Enum**. And extending multiple classes (multiple inheritance) is not allowed in Java.

Java Enum and Interface

As we have learned, we cannot inherit enum classes from other classes in Java. However, enum classes **can** implement interfaces.

[[Handwritten Notes Page 89](#)]

Example: Enum Implementing an Interface

```
interface Pizza {
    public void displaySize();
}

enum Size implements Pizza {
    SMALL, MEDIUM, LARGE, EXTRALARGE;

    @Override
    public void displaySize() {
        System.out.println("The size is " + this);
    }
}

class Main {
    public static void main(String[] args) {
        Size.MEDIUM.displaySize();
    }
}
```

Java Wrapper Classes

Wrapper classes in Java are used to convert primitive types (**int**, **char**, **float**, etc.) into corresponding objects.

Character Wrapper Class

- - **Character** is a wrapper around a **char**.
- - The constructor for **Character** is:

```
Character(char ch)
```

- - Here, **ch** specifies the character that will be wrapped by the **Character** object being created.
- - To obtain the **char** value contained in a **Character** object, call **charValue()**, as shown:

(Continued on Page 90)

[[Handwritten Notes Page 90](#)]

(Continued from Page 89)

Each of the 8 primitive types has corresponding wrapper classes:

| Primitive Type | Wrapper Class |

```
| :--- | :--- |
| byte | Byte |
| boolean | Boolean |
| char | Character |
| double | Double |
| float | Float |
| int | Integer |
| long | Long |
| short | Short |
```

Java Wrapper Types Hierarchy:

```
■■■ Boolean
■■■ Character
■■■ Numeric Types (Number subclasses)
    ■■■ Byte
    ■■■ Short
    ■■■ Integer
    ■■■ Long
    ■■■ Float
    ■■■ Double
```

Example: Manual Wrapping (Boxing & Unboxing)

```
class Wrap {
    public static void main(String args[]) {
        Integer iob = new Integer(100);
        int i = iob.intValue();
        System.out.println(i + " " + iob);
    }
}
```

- - This program wraps the integer value **100** inside an **Integer** object called **iob**.
- - The process of encapsulating a value within an object is called **boxing**:

```
Integer iob = new Integer(100);
```

[Handwritten Notes Page 91]

- - The process of extracting a value from a type wrapper is called **unboxing**:

```
int i = iob.intValue();
```

NOTE 1:

We can also use the **valueOf()** factory method to convert primitive types into corresponding objects.

```
class Main {
    public static void main(String[] args) {
        // Create primitive types
        int a = 5;
        double b = 5.65;

        // Converts into wrapper objects
        Integer aObj = Integer.valueOf(a);
        Double bObj = Double.valueOf(b);

        if (aObj instanceof Integer) {
            System.out.println("An object of Integer is created.");
        }
        if (bObj instanceof Double) {
            System.out.println("An object of Double is created.");
        }
    }
}
```

NOTE 2:

However, the Java compiler can directly convert primitive types into corresponding objects without explicit instantiation or calling `valueOf()` (Autoboxing).

[\[Handwritten Notes Page 92 \]](#)

```
int a = 5;
Integer aObj = a;

double b = 5.6;
Double bObj = b;
```

Autoboxing

Autoboxing in Java is the automatic conversion of a primitive data type to its corresponding wrapper class object.

```
Integer iob = 100; // Autoboxing an int
```

Auto-Unboxing

In this, the Java compiler automatically converts wrapper class objects into their corresponding primitive types.

```
int i = iob; // Auto-unboxing
```

Autoboxing and Methods

Autoboxing and unboxing might occur when an argument is passed to a method or when a value is returned by a method.

Example:

```
class AutoBox2 {
    static int m(Integer v) {
        return v; // Auto-unbox to int
    }

    public static void main(String args[]) {
        Integer iob = m(100); // 100 is autoboxed to Integer, return value is autoboxed to iob
        System.out.println(iob);
    }
}
```

[\[Handwritten Notes Page 93 \]](#)

In the above program:

- 1. **m()** specifies an **Integer** parameter and returns an **int** result.
- 2. Inside **main()**, **m()** is passed the value **100**.
- 3. Because **m()** is expecting an **Integer**, this value is automatically boxed.
- 4. Then, **m()** returns the **int** equivalent of its argument.
- 5. This causes **v** to be auto-unboxed. Next, this **int** value is assigned to **iob** in **main()**, which causes the **int** return value to be autoboxed.

Autoboxing / Unboxing Occurs in Expressions

Within an expression, a numeric object is automatically unboxed. The outcome of the expression is reboxed, if necessary.

Example:

```
class AutoBox3 {
    public static void main(String args[]) {
        Integer iob, iob2;
        int i;

        iob = 100;
        System.out.println("Original value of iob: " + iob);

        /* The following automatically unboxes iob,
           performs the increment, and then
           reboxes the result back into iob */
        ++iob;
        System.out.println("After ++iob: " + iob);
    }
}
```

(Continued on Page 94)

[\[Handwritten Notes Page 94 \]](#)

(Continued from Page 93)

```

        /* Here, iob is unboxed, the expression is evaluated, and
           the result is reboxed and assigned to iob2 */
        iob2 = iob + (iob / 3);
        System.out.println("iob2 after expression: " + iob2);

        /* The same expression is evaluated, but the
           result is not reboxed (assigned to primitive int). */
        i = iob + (iob / 3);
        System.out.println("i after expression: " + i);
    }
}

```

NOTE:

When the **switch** expression is evaluated, **iob** is unboxed and its **int** value is obtained.

```

switch (iob) {
    case 1:
        System.out.println("one");
        break;
    case 2:
        System.out.println("two");
        break;
    case 3:
        System.out.println("three");
        break;
}

```

Autoboxing / Unboxing Boolean and Character Values

```

class AutoBox5 {
    public static void main(String args[]) {
        Boolean b = true;

        // Below, b is auto-unboxed when used in a conditional expression.
        if (b) {
            System.out.println("b is true");
        }
    }
}

```

(Continued on Page 95)

[Handwritten Notes Page 95]

(Continued from Page 94)

```

        Character ch = 'x'; // Autobox a char
        char ch2 = ch;      // Auto-unbox a Character to char
    }
}

```

Autoboxing / Unboxing Helps Prevent Errors

```
class UnboxingError {
    public static void main(String args[]) {
        Integer iob = 1000; // Autobox the value 1000
        int i = iob.byteValue(); // Manually unbox as byte!!!
        System.out.println(i); // Does not display 1000!
    }
}
```

- 1. This program displays not the expected value of **1000**, but **-24**!
- 2. This is because the value inside **iob** is manually unboxed by calling **byteValue()**, which causes the truncation of the value **1000** (since **byte** range is **-128** to **127**).
- 3. This results in the truncated / garbage value of **-24** being assigned to **i**.
- 4. By contrast, autoboxing **iob** will always auto-unbox into a value compatible with **int** (avoiding accidental data truncation).

[[Handwritten Notes Page 96](#)]

Additional String Methods

1. `int codePointAt(int index)`

The method returns the Unicode value of the character at the specified index in a String.

```
String myStr = "Hello";
int result = myStr.codePointAt(0);
System.out.println(result); // Output: 72 (Unicode for 'H')
```

2. `int codePointBefore(int index)`

The **codePointBefore()** method returns the Unicode value of the character before the specified index in a String.

```
String myStr = "Hello";
int result = myStr.codePointBefore(1);
System.out.println(result); // Output: 72
```

Here the index before 1 is 0, so it prints the Unicode value of the character at index 0.

3. `int codePointCount(int startIndex, int endIndex)`

The **codePointCount()** method returns the number of Unicode values found in the specified range of a String.

```
String myStr = "Hello";
int result = myStr.codePointCount(0, 5);
System.out.println(result); // Output: 5
```

4. `boolean contains(CharSequence str)`

The `contains()` method checks whether a String contains a sequence of characters.

```
String myStr = "Hello";
System.out.println(myStr.contains("Hel")); // true
System.out.println(myStr.contains("Hi"));  // false
```

[\[Handwritten Notes Page 97 \]](#)

5. `boolean contentEquals(CharSequence str)`

This method searches a String to find out if it contains the exact same sequence of characters in the specified String or StringBuffer.

```
String myStr = "Hello";
System.out.println(myStr.contentEquals("Hello")); // true
System.out.println(myStr.contentEquals("hello")); // false
System.out.println(myStr.contentEquals("Hi"));    // false
```

6. `String[] split(String regex)`

The `split()` method in Java is used to split a String into an array of substrings based on matches of the given regular expression or specific delimiter.

```
String s = "This is, comma, fullstop, whitespace;";
String regex = "[,\\s.]+";
String[] arr = s.split(regex);
System.out.println(Arrays.toString(arr));
```

Output:

```
[This, is, comma, fullstop, whitespace]
```

NOTE:

In Java, when you try to print an array directly using `System.out.print(arr);` or `System.out.println(arr);`, it does not print the actual elements of the array. Instead, it prints the array's memory reference.

```
[Ljava.lang.String;@1b6d3586
```

Why?

- - In Java, arrays do not override the `toString()` method from `Object`.
- - The default `toString()` from `Object` prints the type and memory address hash, which is not useful for seeing the actual contents.

[\[Handwritten Notes Page 98 \]](#)

Solution:

`Arrays.toString(arr)`:

The `Arrays.toString(arr)` method from `java.util.Arrays` converts the array into a readable String format.

7. `String[] split(String regex, int limit)`

Parameters:

- - **regex**: The regular expression to use as a delimiter.
- - **limit** (Optional): The number of times the pattern is applied and the array size constraint.

Example 1:

```
String s = "geeks@for@geeks";
String[] arr = s.split("@", 2);
for (String a : arr) {
    System.out.println(a);
}
```

Output:

```
geeks
for@geeks
```

Example 2:

```
String s = "geeks@for@geeks";
String[] arr = s.split("@", 5);
for (String a : arr) {
    System.out.println(a);
}
```

Output:

```
geeks
for
geeks
```

8. `String.format()`

```
public static String format(String format, Object... args)
```

- - This method is part of the `String` class and is used to create formatted Strings similar to `printf`.

Example:

```
String name = "Vineeth";
int age = 25;
String formattedStr = String.format("My name is %s and I am %d years old.", name, age);
System.out.println(formattedStr);
```

[\[Handwritten Notes Page 99 \]](#)

11. `boolean matches(String regex)`

Tells whether or not this string matches the given regular expression.

```
String text = "hello";
boolean isMatch = text.matches("hello"); // true
System.out.println(isMatch);

boolean isMatch2 = text.matches("Hello"); // false
System.out.println(isMatch2);
```

12. `String replaceFirst(String regex, String replacement)`

Replaces the first substring of this string that matches the given regular expression with the given replacement string.

```
String text = "apple banana apple";
String newText = text.replaceFirst("apple", "orange");
System.out.println(newText); // Output: "orange banana apple"
```

13. `String replaceAll(String regex, String replacement)`

Replaces each substring of this string that matches the given regular expression with the given replacement string.

```
String text = "apple banana apple";
String newText = text.replaceAll("apple", "orange");
System.out.println(newText); // Output: "orange banana orange"
```

14. `CharSequence subSequence(int startIndex, int stopIndex)`

Returns a [CharSequence](#) that is a subsequence of this sequence.

```
String text = "Hello, World!";
CharSequence sub = text.subSequence(7, 12);
System.out.println(sub); // Output: "World"
```

[\[Handwritten Notes Page 100 \]](#)

StringBuffer

- 1. [StringBuffer](#) supports a modifiable (mutable) String.
- 2. As you know, [String](#) represents fixed-length, immutable character sequences.
- 3. In contrast, [StringBuffer](#) represents growable and writable character sequences.

StringBuffer Constructors

1. `StringBuffer()`

```
StringBuffer s = new StringBuffer();
```

- - Creates an empty string buffer with initial capacity of 16 characters.
- - It reserves room for 16 characters without reallocation.

2. `StringBuffer(int size)`

```
StringBuffer s = new StringBuffer(20);
```

- - It accepts an integer argument that explicitly sets the initial capacity (size) of the buffer.

3. `StringBuffer(String str)`

```
StringBuffer s = new StringBuffer("GeeksforGeeks");
```

- - It accepts a **String** argument that sets the initial contents of the **StringBuffer** object and reserves room for 16 more characters without reallocation (**length + 16**).

NOTE:

If you allocate the size to the **StringBuffer** and if you give/append more characters than its initial capacity, it automatically re-allocates and expands the size.

[[Handwritten Notes Page 101](#)]

StringBuffer: Methods Continued

1. `length()` and `capacity()`

- * **length()**: Returns the number of characters currently present in the buffer.
- * **capacity()**: Returns the total allocated memory space for characters (default initial capacity is 16 + length of initial string).

```
StringBuffer sb = new StringBuffer("Hello");

System.out.println(sb.length()); // Output: 5 -> length
System.out.println(sb.capacity()); // Output: 21 -> capacity (16 + 5)
```

2. `ensureCapacity()`

Ensures that the buffer capacity is at least equal to the specified minimum capacity. If the current capacity is less, the buffer expands.

```
StringBuffer sb = new StringBuffer();
System.out.println(sb.capacity()); // Output: 16

sb.ensureCapacity(50);
System.out.println(sb.capacity()); // Output: 50
```

3. `setLength()`

```
public void setLength(int newLength)
```

This method sets the length of the character sequence inside the **StringBuffer**.

Case 1: Truncating

If **newLength** is smaller than the current **length()**, the sequence is truncated.

```
StringBuffer sb = new StringBuffer("Hello world");
sb.setLength(5);
System.out.println(sb); // Output: Hello
```

Case 2: Expanding

If **newLength** is greater than the current **length()**, null characters (**'\u0000'**) are appended to fill the extra length.

```
StringBuffer sb = new StringBuffer("Hi");
sb.setLength(5);

System.out.println(sb.length()); // Output: 5
System.out.println(sb);           // Output: "Hi" + 3 null characters (invisible)
```

[[Handwritten Notes Page 102](#)]

StringBuffer: Common Methods

1. `charAt(int index)`

Reads the character at the specified index.

```
StringBuffer sb = new StringBuffer("Hello");
char ch = sb.charAt(1);
System.out.println(ch); // Output: 'e'
```

2. `setCharAt(int index, char ch)`

Modifies the character at the specified index.

```
StringBuffer sb = new StringBuffer("Hello");
sb.setCharAt(1, 'a');
System.out.println(sb); // Output: Hallo
```

3. `getChars(int srcBegin, int srcEnd, char[] dst, int dstBegin)`

Copies characters from this `StringBuffer` into the destination character array `dst`.

```
StringBuffer sb = new StringBuffer("HelloWorld");
char[] arr = new char[5];
sb.getChars(0, 5, arr, 0);
System.out.println(arr); // Output: Hello
```

4. `append(...)`

Adds data to the end of the buffer.

```
StringBuffer sb = new StringBuffer("Hello");
sb.append(" World");
System.out.println(sb); // Output: Hello World
```

5. `insert(int offset, ...)`

Inserts data at the specified index.

```
StringBuffer sb = new StringBuffer("Hello");
sb.insert(2, 'l'); // inserts 'l' at index 2
System.out.println(sb); // Output: Hello
```

6. `reverse()`

Reverses the sequence of characters in the buffer.

```
StringBuffer sb = new StringBuffer("Hello");
sb.reverse();
System.out.println(sb); // Output: olleH
```

7. `deleteCharAt(int index)`

Deletes the character at the specified index.

```
StringBuffer sb = new StringBuffer("Hello");
sb.deleteCharAt(1);
System.out.println(sb); // Output: Hllo
```

[\[Handwritten Notes Page 103 \]](#)

StringBuffer: Common Methods

> **Note:** Page 103 is a duplicate scan of the notes on Page 102.

1. `charAt(int index)`

Reads the character at the specified index.

```
StringBuffer sb = new StringBuffer("Hello");
char ch = sb.charAt(1);
System.out.println(ch); // Output: 'e'
```

2. `setCharAt(int index, char ch)`

Modifies the character at the specified index.

```
StringBuffer sb = new StringBuffer("Hello");
sb.setCharAt(1, 'a');
System.out.println(sb); // Output: Hallo
```

3. `getChars(int srcBegin, int srcEnd, char[] dst, int dstBegin)`

Copies characters from this **StringBuffer** into the destination character array **dst**.

```
StringBuffer sb = new StringBuffer("HelloWorld");
char[] arr = new char[5];
sb.getChars(0, 5, arr, 0);
System.out.println(arr); // Output: Hello
```

4. `append(...)`

Adds data to the end of the buffer.

```
StringBuffer sb = new StringBuffer("Hello");
sb.append(" World");
System.out.println(sb); // Output: Hello World
```

5. `insert(int offset, ...)`

Inserts data at the specified index.

```
StringBuffer sb = new StringBuffer("Helo");
sb.insert(2, 'l'); // inserts 'l' at index 2
System.out.println(sb); // Output: Hello
```

6. `reverse()`

Reverses the sequence of characters in the buffer.

```
StringBuffer sb = new StringBuffer("Hello");
sb.reverse();
System.out.println(sb); // Output: olleH
```

7. `deleteCharAt(int index)`

Deletes the character at the specified index.

```
StringBuffer sb = new StringBuffer("Hello");
sb.deleteCharAt(1);
System.out.println(sb); // Output: Hllo
```

[\[Handwritten Notes Page 104 \]](#)

StringBuffer & Introduction to StringBuilder

`replace(int start, int end, String str)`

Replaces the characters in a substring of this sequence with characters in the specified **String**.

```
StringBuffer sb = new StringBuffer("Hello Java");
sb.replace(6, 10, "World");
System.out.println(sb); // Output: Hello World
```

`substring(...)`

- * **substring(int start)**: Returns a new **String** that contains a subsequence of characters starting from **start** index to the end.
- * **substring(int start, int end)**: Returns a new **String** containing characters from index **start** up to index **end - 1**.

(Known / standard String operation)

StringBuilder (Not Thread Safe)

- * Introduced in Java 5 to provide a high-performance alternative to **StringBuffer**.
- * **Key Difference from StringBuffer**:
- * **StringBuffer**: Thread-safe (methods are synchronized), with synchronization overhead.
- * **StringBuilder**: **Not** thread-safe (methods are not synchronized), significantly faster and preferred for single-threaded string manipulations.

[[Handwritten Notes Page 105](#)]

Nested Classes

What is a Nested Class?

A **nested class** is a class written inside another enclosing class.

```
class Outer {  
    class Inner {  
    }  
}
```

Why did Java introduce Nested Classes?

> **"Some classes exist only to support one specific class."**

Instead of keeping them outside and exposing them to the whole world, Java allows helper classes to be placed inside the class they belong to.

Why does Java allow this?

1. Logical Grouping

Problem without Nested Classes:

```
class Engine {
    void start() {}
}

class Car {
    Engine e = new Engine();
}
```

- * **Issues:**
- * **Engine** is visible everywhere across the package/application.
- * Anyone can misuse or instantiate it independently.
- * Logically, an **Engine** belongs to a **Car**.

With Nested Classes:

```
class Car {
    class Engine {
        void start() {}
    }
}
```

- * **Now:**
- * **Engine** is logically grouped inside **Car**.
- * Anyone reading the code immediately knows: `Engine is part of Car`.

[[Handwritten Notes Page 106](#)]

Advantages of Nested Classes: Encapsulation & Security

2. Better Encapsulation

- * **Encapsulation** = Hiding internal implementation details.

```
class BankAccount {
    private double balance;

    private class Logger {
        void log(String msg) {
            System.out.println(msg);
        }
    }

    void deposit(double amount) {
        balance = balance + amount;
        new Logger().log("Amount deposited");
    }
}
```

What is happening?

- * **Logger** is declared **private**.
- * Only **BankAccount** can access and use it.
- * The outside world cannot even see or instantiate **Logger**.

3. Security

Rule:

- * Nested classes can access **private members** of the outer class.
- * The outer class can access **private members** of the nested class.

Example:

```
class PasswordManager {
    private String password = "Secret123";

    private class PasswordChecker {
        boolean check() {
            return password.length() > 8;
        }
    }

    boolean isStrong() {
        return new PasswordChecker().check();
    }
}
```

What security is happening here?

- * **password** is **private**.
- * Only **PasswordManager** and its nested helper class can touch it.
- * No other external class can read, modify, or create passwords.
- * **Key Takeaway:** The nested class acts like a **trusted internal employee**.

[\[Handwritten Notes Page 107 \]](#)

Types of Nested Classes (4 Types)

1. Non-Static Inner Class (Member Inner Class)

```
class Outer {
    int x = 10;

    class Inner {
        void show() {
            System.out.println(x);
        }
    }
}
```

Key Characteristics:

- * Can access all members (both static and non-static/private) of the outer class.

- * Cannot declare static members (prior to Java 16).
- * Requires an outer class object to be instantiated.

How to create an object:

```
Outer outer = new Outer();
Outer.Inner inner = outer.new Inner();
inner.show();
```

2. Static Nested Class

A static class defined inside another class.

```
class Outer {
    static int x = 20;

    static class Inner {
        void show() {
            System.out.println(x);
        }
    }
}
```

Key Characteristics:

- * Can access **only static** members of the outer class directly.
- * No outer class object is required for instantiation.
- * Can declare static members.
- * Less memory overhead than an inner class (no reference pointer to outer instance).

Object Creation:

```
Outer.Inner obj = new Outer.Inner();
obj.show();
```

Differences: Inner Class vs Static Nested Class

Feature	Inner Class (Non-Static)	Static Nested Class
Instantiation	Outer obj = new Outer(); obj.new Inner();	Outer.Inner obj = new Outer.Inner();
Needs Outer Object?	Yes	No
Access Outer Variables	All (Instance & Static)	Only Static
Memory Overhead	More (holds pointer to outer instance)	Less (no outer pointer)
Can have Static members?	X *(prior to Java 16)*	✓

[\[Handwritten Notes Page 108 \]](#)

Types of Nested Classes: Local & Anonymous Inner Classes

3. Local Inner Class

A class defined inside a method.

```
class Outer {  
    void display() {  
        class LocalInner {  
            void show() {  
                System.out.println("Local Inner");  
            }  
        }  
  
        LocalInner obj = new LocalInner();  
        obj.show();  
    }  
}
```

Rules:

- * Can access method local variables **only if they are `final` or effectively final**.
- * Scope is strictly limited to the enclosing method.
- * Used rarely in modern Java.

4. Anonymous Inner Class

An **anonymous inner class** is an inner class without a name that is created at the time of object instantiation and used only once.

Concept:

Instead of:

- 1. Creating a separate named class (**`class MyHandler implements ...`**)
- 2. Giving it a name
- 3. Then creating an object of it

We combine all of these steps into a single concise expression!

[\[Handwritten Notes Page 109 \]](#)

Anonymous Inner Class: Example & Behind the Scenes

Example:

```
class Parent {
    void show() {
        System.out.println("Parent Show");
    }
}

class Demo {
    public static void main(String[] args) {
        Parent p = new Parent() {
            @Override
            void show() {
                System.out.println("Overridden in Anonymous class");
            }
        };
        p.show();
    }
}
```

Output:

```
Overridden in Anonymous class
```

Behind the Scenes: What Java Does Internally

Behind the scenes, the Java compiler automatically creates an anonymous helper class file (e.g., **Demo\$1.class**):

```
// Compiler-generated class:
class Demo$1 extends Parent {
    @Override
    void show() {
        System.out.println("Overridden in Anonymous class");
    }
}
```

And then rewrites the instantiation as:

```
Parent p = new Demo$1();
```

Summary:

- * Java automatically generates a subclass of **Parent**.
- * That subclass has no programmer-defined name in code (hence called **Anonymous**).
- * It extends **Parent** (or implements an interface) automatically.

[[Handwritten Notes Page 110](#)]

JVM Memory Model

The **JVM Memory Model** defines how Java manages memory at runtime and where different kinds of data live.

JVM Memory Areas (5 Main Areas)

The JVM divides runtime memory into 5 main areas:

```
JVM Memory
■■■ Metaspace (Method / Class Area)
■■■ Heap
■■■ Stack
■■■ PC Register (Program Counter)
■■■ Native Method Stack
```

Summary of Memory Areas

Memory Area	Stores	Lifetime
Metaspace	Class metadata, bytecodes, static variables	Until class is unloaded
Heap	All Objects and instance variables	Until Garbage Collected (GC)
Stack	Stack frames, method calls, local variables & references	Until method execution ends
PC Register	Address of the current JVM instruction being executed	Per thread
Native Stack	Native method calls & C/C++ execution state	Until native call ends

| :--- | :--- | :--- |

| **Metaspace** | Class metadata, bytecodes, static variables | Until class is unloaded |

| **Heap** | All Objects and instance variables | Until Garbage Collected (GC) |

| **Stack** | Stack frames, method calls, local variables & references | Until method execution ends |

| **PC Register** | Address of the current JVM instruction being executed | Per thread |

| **Native Stack** | Native method calls & C/C++ execution state | Until native call ends |

What is the Native Stack?

- * **Native Stack** is the stack memory used when **native (C/C++) methods** are executed.
- * It is used when Java code interacts with another programming language via JNI (Java Native Interface).

[[Handwritten Notes Page 111](#)]

JNI (Java Native Interface) & Native Method Stack

Example: Java Native Method

```
class HelloNative {
    // Native method declaration (no body)
    public native void hello();

    static {
        // Loads native library (e.g., hello.dll on Windows or libhello.so on Linux)
        System.loadLibrary("hello");
    }

    public static void main(String[] args) {
        HelloNative obj = new HelloNative();
        obj.hello();
        System.out.println("Back in Java");
    }
}
```

=> **hello()** has no body - JVM must go to native code via Native Method Stack.

Corresponding C Code:

```
#include <jni.h>
#include <stdio.h>
#include "HelloNative.h"

JNIEXPORT void JNICALL Java_HelloNative_hello(JNIEnv *env, jobject obj) {
    printf("Hello from C\n");
}
```

Notes on JNI Signature:

- * **JNIEXPORT void JNICALL Java_HelloNative_hello(...)**: C function that implements the Java **hello()** method of class **HelloNative**.
- * **JNIEnv *env**: Pointer to JNI environment table; helps C code interact with the Java Virtual Machine.
- * **jobject obj**: Reference pointing to the **HelloNative** Java object that invoked the native method.

[[Handwritten Notes Page 112](#)]

JVM Memory Model: Comprehensive Code Example (Part 1)

This comprehensive example illustrates where each component resides in the JVM memory areas (Metaspace, Heap, Stack).

```
enum Level {
    LOW, MEDIUM, HIGH
}

class Outer {
    static int s = 10;
    int x = 20;

    static void staticMethod() {
        System.out.println("Static Method");
    }

    void instanceMethod() {
        int local = 30;

        class LocalInner {
            void show() {
                System.out.println(s);
                System.out.println(x);
                System.out.println(local);
            }
        }

        LocalInner li = new LocalInner();
        li.show();

        // Anonymous class
        Runnable r = new Runnable() {
            @Override
            public void run() {
                System.out.println("Anonymous class run()");
            }
        };
        r.run();
    }
}
```

(Code continues on the next page...)

[\[Handwritten Notes Page 113 \]](#)

JVM Memory Model: Comprehensive Code Example (Part 2)

(Continuation of Outer class and driver program)

```
// Static nested class
static class StaticInner {
    static int si = 40;

    void show() {
        System.out.println(s);
    }
}

// Non-static inner class
class Inner {
    void show() {
        System.out.println(x);
        System.out.println(s);
    }
}

public class JVMAllInOneDemo {
    public static void main(String[] args) {
        Outer o = new Outer();
        o.instanceMethod();

        Outer.StaticInner si = new Outer.StaticInner();
        si.show();

        Outer.Inner i = o.new Inner();
        i.show();

        Level level = Level.HIGH;
        System.out.println(level);
    }
}
```

[\[Handwritten Notes Page 114 \]](#)

JVM Memory Execution Trace: Step 1 (Metaspace)

Step 1: METASPACE Loading

Before **main()** runs, the JVM ClassLoader loads all required **.class** files into **Metaspace**.

```

Metaspace
████ JVMAllInOneDemo.class
█   █████ main(String[] args)
████ Outer.class
█   █████ static int s = 10           <-- Static variable value
█   █████ field metadata: int x       <-- Field metadata only (instance value not stored here)
█   █████ staticMethod()
█   █████ instanceMethod()
█   █   █████ LocalInner.class (Outer$1LocalInner.class)
█   █   █   █████ show()
█   █   █████ Anonymous Runnable (Outer$1.class)
█   █   █   █████ run()
█   █████ class Inner (Outer$Inner.class)
█   █   █████ show()
█   █████ static class StaticInner (Outer$StaticInner.class)
█   █████ static int si = 40
█   █████ show()
████ Level.class (enum)
    █████ LOW
    █████ MEDIUM
    █████ HIGH
    
```

[\[Handwritten Notes Page 115 \]](#)

JVM Memory Execution Trace: Steps 2 to 5

Step 2: Enum Initialization

```
enum Level { LOW, MEDIUM, HIGH }
```

What JVM does internally:

- * Creates 3 **public static final** objects of type **Level**.
- * Each enum constant is an **Object!**
- * Enum objects are created during class loading and stored in the **Heap**.

```

Heap
████ Level.LOW
████ Level.MEDIUM
████ Level.HIGH
    
```

> Note:

- > * Enum metadata & bytecodes reside in **Metaspace**.
- > * Enum constant objects reside in the **Heap**.

Step 3: `main()` Execution Begins

```
public static void main(String[] args)
```

- * A stack frame for **main()** is pushed onto the thread's **Stack**.

```
Stack
■■■ [Frame: main()]
```

Step 4: Object Creation

```
Outer o = new Outer();
```

- * Variable reference **o** is created in **main()** stack frame.
- * The **Outer** object is allocated in the **Heap**.

```
Stack
■■■ o ■■■■■■■■■■■■■■■■■■■■■■
                        ▼
Heap
■■■ Outer object
    ■■■ x = 20
```

Step 5: Instance Method Call

```
o.instanceMethod();
```

- * A new stack frame for **instanceMethod()** is pushed onto the **Stack**.
- * It stores **this** pointer pointing to the **Outer** object, and the local variable **local = 30**.

```
Stack
■■■ [Frame: main()]
    ■■■ [Frame: instanceMethod()]
        ■■■ this  ■■> Outer object in Heap
        ■■■ local = 30
```

[[Handwritten Notes Page 116](#)]

JVM Memory Execution Trace: Steps 6 to 9

Step 6: Local Inner Class Object Creation

```
LocalInner li = new LocalInner();
```

- * **li** object is created in the **Heap**.
- * It contains an implicit reference to the enclosing **Outer** instance and a copy/captured copy of **local = 30**.

```
Heap
■■■ LocalInner object
    ■■■ reference to Outer object
    ■■■ captured local = 30
```

Step 7: Anonymous Inner Class Object Creation

```
Runnable r = new Runnable() { ... };
```

- * The anonymous class bytecodes are loaded in **Metaspace**.
- * The anonymous class object is allocated in the **Heap**.

```
Metaspace
■■■ Outer$1.class (implements Runnable)
    ■■■ run()

Heap
■■■ Outer$1 object (Anonymous instance)
```

Step 8: Static Inner Class Object Creation

```
Outer.StaticInner si = new Outer.StaticInner();
```

- * Allocated in the **Heap**.

```
Heap
■■■ StaticInner object
```

> **Note:**

- > * Contains **NO reference** to the enclosing **Outer** class instance.
- > * Can only access static members of the enclosing class.

Step 9: Non-Static Inner Class Object Creation

```
Outer.Inner i = o.new Inner();
```

- * Allocated in the **Heap**.

```
Heap
■■■ Inner object
    ■■■ reference to Outer object (outer instance pointer)
```

[[Handwritten Notes Page 117](#)]

JVM Memory Execution Trace (Step 10) & Concept of Static

Step 10: Enum Usage

```
Level level = Level.HIGH;
```

- * The local reference variable **level** is placed in the **main()** stack frame.
- * It points to the already existing **Level.HIGH** object created on the **Heap**.



— — —

Concept of Static: How Object References Resolve Static Members

> Question:

> Static members belong to the class, not the object. Then how can we call static members using an object reference variable (e.g., `t.show()`)?

Answer:

Static members are resolved at **compile-time** using the **REFERENCE TYPE**, not the actual runtime object!

Even though you write:

```
t.show();
```

The compiler checks the declared type of **t** (**Test t**), and rewrites it internally to:

```
Test.show();
```

Example:

```
class Test {
    static int x = 10;

    static void show() {
        System.out.println("Hello");
    }
}

public class Main {
    public static void main(String[] args) {
        Test t = new Test();
        t.show(); // Compiler converts to Test.show()
        System.out.println(t.x); // Compiler converts to System.out.println(Test.x)

        Test tNull = null;
        tNull.show(); // Still works! No NullPointerException because
                     // compiler resolves it to Test.show() using reference type.
    }
}
```

- Even if `t == null`, `t.show()` will still work without throwing `NullPointerException`, because the compiler translates it directly to `Test.show()` at compile time based purely on `t`'s declared type.